KTH

DEGREE PROJECT IN COMPUTER SCIENCE AND ENGINEERING
SECOND CYCLE, 30 CREDITS

Stockholm, Sweden 2026

Joint Radar Emitter and Mode Clustering Using a Transformer Encoder Architecture

Markus Swegmark

Supervisor: TBD
Examiner: TBD
Host: TBD

KTH Royal Institute of Technology
School of Electrical Engineering and Computer Science
Master's Programme in Machine Learning

TRITA-EECS-EX-XXXX:XXX

Abstract

This thesis investigates joint clustering of radar emitters and their operating modes from intercepted pulse descriptor word (PDW) streams using a transformer encoder architecture.

Keywords: transformer, deep clustering, electronic intelligence, radar emitter identification, mode recognition, supervised contrastive learning.

Sammanfattning

Detta examensarbete undersöker gemensam klustring av radarsändare och deras operativa moder från avlyssnade pulsbeskrivande ord (PDW) med hjälp av en transformerbaserad encoderarkitektur.

Nyckelord: transformer, djup klustring, signalspaning, radarsändaridentifiering, modigenkänning, självövervakad inlärning.

Acknowledgments

I would like to thank my supervisor and examiner at KTH, and the host organization for their support throughout this thesis project.

Stockholm, August 2026
Markus Swegmark

Contents

List of Figures

List of Tables

List of Acronyms

AMI adjusted mutual information

AOA angle of arrival

ARI adjusted Rand index

BERT Bidirectional Encoder Representations from Transformers

CNN convolutional neural network

DBSCAN Density-based spatial clustering of applications with noise

DEC deep embedded clustering

DL deep learning

ELINT electronic intelligence

ESM electronic support measures

FFN feed-forward network

GMM Gaussian mixture model

IDEC improved deep embedded clustering

MHA multi-head attention

ML machine learning

MLP multilayer perceptron

PDW pulse descriptor word

PRF pulse repetition frequency

PRI pulse repetition interval

RADAR radio detection and ranging

RF radio frequency

RNN recurrent neural network

RoPE rotary positional encoding

TOA time of arrival

V-measure harmonic mean of homogeneity and completeness

CHAPTER 1

Introduction

1.1 Background and Motivation

An electromagnetic wave carries electric and magnetic fields through space at a known speed. A conducting body in its path scatters some of that energy back toward the source. In 1904 Christian Hülsmeyer used that return to detect ships, decades before the method was named RADAR and used to measure range [1]. A transmitter that illuminates a scene also announces itself to anyone in range if they are capable of receiving the signal.

Passive ESM receivers do not transmit. They record a time-ordered stream of PDWs and feed ELINT analysis: detecting radars, grouping pulses by emitter, and recognizing operating modes [2, 3]. The listener now faces a mixture, not a single echo. Radar and communications share a limited spectrum, and several emitters occupy it at once.

Modern radars vary carrier frequency, pulse width, amplitude, and PRI from burst to burst. Fingerprints built on a stable pulse train then weaken, and parsers written for regular trains become less reliable. The same emitter also switches mode, a recurrent control policy that maps a task such as volume search or track to a characteristic pulse regime. Knowing the mode constrains how the waveform may evolve, and it can separate emitters that look similar at the type level.

Classical pipelines still parse these streams with rules written for regular trains. When several agile emitters occupy the same window, the parser has to read context rather than pulses in isolation. The supervision available in labelled training data is asymmetric: emitter identifiers are unique only within a recording, while operating modes come from a catalogue shared across recordings.

This thesis asks whether a transformer encoder, trained with clustering objectives, can map such streams to embeddings in which emitters and modes both form separable groups.

1.2 Problem Statement

Passive ESM produces a time-ordered stream of PDWs in which pulses from several emitters are interleaved and each emitter may change mode over time (Section 1.1). Operators need both a grouping by emitter and a grouping by mode from the same stream. Classical pipelines usually deinterleave first and then apply separate mode heuristics or classifiers. That split does not produce one representation whose geometry shows both partitions at once, and it scales poorly when waveforms are agile and pulse

trains overlap.

This thesis treats the two partitions as a joint learning problem. Training data come from a scenario simulator (Section 4.2) in which every pulse has two labels: an emitter identity that is unique only inside its recording, and a mode drawn from a catalogue shared across recordings. The encoder maps each window of pulses to two embedding sequences. A clustering step on each sequence recovers the partitions, without assigning names from a fixed catalogue. Emitter clustering is recording-local and does not assume a known number of emitters. Mode labels are used in training to pull same-mode pulses together; at inference, mode clusters are also formed per window. The experiments measure agreement with the simulator partitions under mixture, noise, and a fixed window length.

1.3 Research Questions

The thesis addresses the following questions:

RQ1 Given a bounded window of PDWs, can a transformer encoder deinterleave pulses and recover operating modes by clustering?

RQ2 Does a joint objective that couples deinterleaving with mode clustering outperform a comparable transformer trained for deinterleaving alone?

RQ3 Do a pairwise physical attention bias and rotary positional encoding on pulse coordinates improve clustering relative to standard multi-head attention?

1.4 Objectives and Scope

1.4.1 Objectives

We implement a transformer encoder that maps windows of PDWs to emitter and mode embeddings, cluster each embedding independently, and measure agreement with simulator labels using AMI, ARI, and V-measure (Sections 4.1 and 4.3 and Chapter 5). We train with two supervised contrastive terms—recording-local emitter positives and batch-wide mode positives—and compare that joint loss with a deinterleaving-only baseline of similar capacity (Section 4.5 and Chapter 5). We compare standard self-attention with a physical pairwise bias and with rotary positional encoding on pulse coordinates (Section 4.4 and Chapter 5).

1.4.2 Delimitations

The model sees PDWs, not raw radio-frequency samples. Pulse detection, receiver processing, and multi-platform fusion are out of scope.

All data come from a proprietary scenario simulator (Section 4.2). No operational ELINT recordings are used, so the results do not speak to real-world distribution shift. Dropped pulses and spurious detections, when present, are simulator artifacts.

The study is offline analysis of recorded streams, not real-time inference on embedded hardware. The encoder does not name emitters against a classified library. Self-attention scales with the square of window length, so each recording is split into windows of fixed length (Section 4.2.2). Scores therefore describe clustering on those windows, not tracking over an unbounded stream.

Every evaluation scenario uses the same mode catalogue as training (Section 5.2). Held-out modulation types are not tested.

1.5 Contributions

The main contributions of this thesis are:

- We propose a transformer encoder with a shared trunk and two branches that map a window of PDWs to emitter and mode embeddings, decoded by per-window DBSCAN (Section 4.3).
- We train both branches with supervised contrastive losses that differ only in the scope of the positive set: recording-local emitter identifiers versus global mode labels (Section 4.5).
- We evaluate a pairwise physical attention bias and a rotary positional encoding on time and azimuth as drop-in changes to standard self-attention (Section 4.4).
- We test the method on a labelled synthetic scenario corpus and report emitter and mode clustering with AMI, ARI, and V-measure (Chapter 5).

1.6 Ethical and Societal Considerations

Methods for passive ESM and ELINT sit squarely in dual-use territory: the same representations that support defensive situational awareness, spectrum management, and safety-of-navigation applications can, in principle, support targeting or surveillance against specific platforms if deployed without appropriate policy, oversight, and legal constraints. This thesis develops ML techniques on abstract pulse-level features and does not address deployment, rules of engagement, or classification of live operational data; nevertheless, the capability to group emitters and recognise operating modes from PDW streams should be understood as a general-purpose building block whose societal consequences depend on the institutional context in which it is used.

From a data-ethics perspective, the experimental work relies on controlled or synthetic PDW streams rather than collections derived from live emitters, which limits

privacy and classification sensitivity concerns at the cost of weaker guarantees about behavior under classified or vendor-specific waveforms. When such methods are extended toward operational data, stewards must enforce access control, purpose limitation, and retention policies consistent with national regulation and organizational policy, and they should document provenance and limitations so that operators do not over-trust cluster labels that lack ground truth.

Training modern DL encoders incurs non-negligible energy use for computation and cooling; relative to lightweight classical deinterleaving heuristics, transformer-scale models increase the carbon footprint of experimentation and of any future always-on training loops. Mitigation follows standard practice: reuse checkpoints, avoid redundant hyperparameter sweeps, prefer smaller models when they meet accuracy targets, and report approximate training cost where it informs reproducibility.

At a societal level, automation of emitter and mode analysis may concentrate analytic capacity in well-resourced actors, widen asymmetries in electromagnetic situational awareness, and reduce the visibility of human expert judgment if clusters are treated as authoritative without audit trails. Responsible research therefore favors transparent evaluation protocols, clear statements of failure modes under noise and mixture, and conservative claims about operational readiness.

1.7 Thesis Outline

Chapter 2 covers pulsed radar, PDWs, emitters and modes, the ELINT chain, cluster agreement scores, and the machine-learning and transformer background used later. Chapter 3 surveys classical emitter identification, supervised radar models, deep clustering, sequence transformers, and joint clustering, and states the gap that motivates the method. Chapter 4 defines the problem, the preprocessing, the encoder and losses, the attention variants, training, the choice of scores, and baselines. Chapter 5 reports setup, the synthetic corpus, hyperparameters, scores, ablations, and qualitative plots. Chapter 6 interprets the results, compares them with prior work, and states limitations. Chapter 7 answers the research questions and lists future work.

CHAPTER 2

Background

2.1 Radar Systems and Passive Reception

Pulsed radar

A radar transmits a short electromagnetic pulse, waits for the echo that scatters from objects in the environment, and infers range and motion from that return [3].

Distance follows from the round-trip delay τ between transmission and reception. With wave speed c, the range of a monostatic radar (transmitter and receiver on the same platform) is

$$R = \frac{c\tau}{2}, \tag{2.1}$$

where the factor of 2 accounts for the outbound and return paths [3]. Bistatic and multistatic geometries place the receive antenna elsewhere and add angle of arrival and other spatial measurements to this range estimate.

Radars emit pulses at a PRI, or equivalently a PRF. If an echo from an earlier pulse arrives after the next pulse has been sent, the measured delay no longer identifies a unique range. This is called range ambiguity. The largest range that remains unique is the unambiguous range

$$R_{\mathrm{u}} = \frac{c}{2\,\mathrm{PRF}} = \frac{c\,\mathrm{PRI}}{2}. \tag{2.2}$$

Matched filtering and coherent integration improve detection inside this window, but they do not remove the ambiguity imposed by the pulse timing [3].

Active radar versus passive reception

An active radar times the echo against its own outgoing pulses and from that delay obtains range [3]. The same pulses are visible to any receiver in range, so transmission also discloses presence, location, and activity [2].

Passive systems such as ESM and ELINT do not transmit. They listen to other radars to determine which emitters are present and how those emitters are behaving [2]. Without control of the waveform, the receiver is limited to the quantities it can measure after detection. Those measurements are stored as the PDWs defined next.

2.1.1 Pulse descriptor words

A receiver records a voltage time series at the antenna. Pulses appear as brief rises in energy above the noise floor. For each detection the receiver estimates carrier frequency,

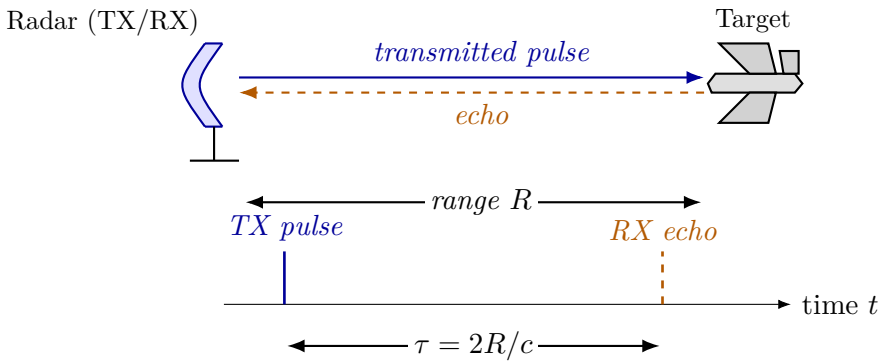


Figure 2.1. Monostatic radar geometry and round-trip timing. A co-located transmit/receive (TX/RX) aperture on the left emits a pulse that travels to a target at range R, scatters, and returns to the same aperture as an echo. The lower timeline shows the corresponding events: the outgoing TX pulse at $t = 0$ and the received echo at $t = \tau$, separated by the round-trip delay $\tau = 2R/c$ from Equation (2.1).

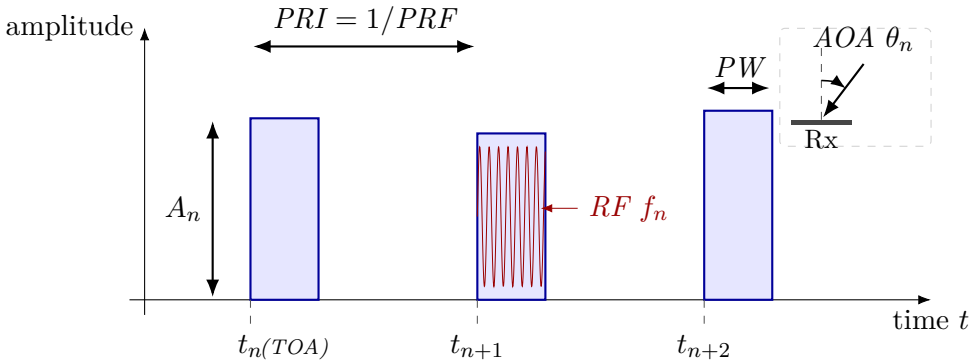


Figure 2.2. Parameters captured by a PDW for a short pulse train. Each pulse contributes a TOA (the leading edge t_n on the time axis), a pulse width (PW), a peak amplitude A_n, and a carrier RF f_n, shown as the fast oscillation inside the middle pulse. Successive pulses also define a PRI (equivalently a PRF $= 1/\text{PRI}$). The inset shows the AOA θ_n measured at the receive aperture relative to a boresight reference. A typical PDW stacks these per-pulse measurements into a vector $\boldsymbol{x}_n = (f_n, \text{PW}_n, t_n, A_n, \theta_n)$ [2].

pulse width, time of arrival, amplitude, and, when directional antennas are available, angle of arrival.

Those estimates are stored as a PDW:

$$\boldsymbol{x}_n = \left(f_n,\ \text{PW}_n,\ t_n,\ A_n,\ \theta_n,\ \ldots\right)^{\top} \in \mathbb{R}^d, \tag{2.3}$$

where f_n is the measured carrier frequency, PW_n is pulse width, t_n is time of arrival (often the leading edge), A_n is amplitude, and θ_n is angle of arrival if available. Further entries, such as elevation, may be included, but once the template is chosen every pulse is stored in the same format. Figure 2.2 shows this five-parameter template.

A sequence of pulses from one transmitter is a *pulse train*. When that transmitter

remains in one mode, the inter-arrival times

$$\tau_n = t_n - t_{n-1}, \qquad n \geq 2 \tag{2.4}$$

are regular or follow a programmed schedule. That interval is the PRI of the train, and its inverse is the PRF. Simple emitters keep a fixed interval. Agile emitters vary it in a planned way.

Each measured descriptor is a noisy, possibly incomplete observation of the true pulse. Weak pulses may be missed, and noise events may be reported as pulses.

After this step the sampled voltage is discarded. The input to later stages is a time-ordered sequence of PDWs $(\boldsymbol{x}_1, \ldots, \boldsymbol{x}_T)$, called a *recording* because it generally interleaves the pulse trains of several transmitters.

2.1.2 Emitters and operating modes

In passive ESM and ELINT, an *emitter* is a physical transmitter: pulses that share timing, frequency, and direction closely enough to be treated as one source [2]. An *operating mode* is the transmit schedule in force at a given time: carrier plan, pulse width, repetition pattern, and scan or switching rules, chosen for a task such as search, acquisition, track, or illumination [2, 3].

These are two different groupings of the same pulses. One emitter switches among modes as its task changes, so a single physical source produces several mode segments over a mission. Different emitters can also run the same functional mode with different numerical parameters, for example two trackers with distinct nominal PRFs [2].

The classical chain therefore has to answer both: which transmitter is present, and what that transmitter is doing now. Section 2.2 describes how it does so in sequence.

2.2 The Classical ELINT Pipeline

A passive collector produces a time-ordered list of PDWs and then has to answer two questions about that list: which physical emitter sent each pulse, and which operating mode was in use. Classical ELINT and ESM architectures answer those questions in stages rather than jointly [2]. Implementations differ by band and platform, but the order of work is the same: detect and measure, deinterleave into pulse trains, then analyze mode on each train.

Detection and pulse measurement

Antennas and RF conditioning capture a band of interest. A detector marks candidate pulses against noise and clutter. Parameter estimation then attaches the per-pulse measurements of Section 2.1.1, producing the PDW list that later stages consume.

Estimates are noisy, weak pulses may be missed, and false alarms may enter the list with no associated radar.

The list is still mixed. Pulses from several emitters, possibly in several modes, share one time-ordered recording.

2.2.1 Deinterleaving: grouping pulses by emitter

Deinterleaving, also called sorting, assigns each pulse to a physical emitter. Its output is a set of pulse trains, one hypothesized sequence per emitter [2]. This recovers the emitter grouping of Section 2.1.2, not a name from a catalogue.

When trains are sparse and nearly periodic, the PRI inferred from TOA differences is a usable timing signature. Histogram methods such as CDIF and SDIF accumulate inter-pulse intervals and treat peaks as candidate PRIs [2, 4]. They fail when timing is agile, and when missing pulses break sequential differencing. Overlapping trains are a further problem, because differences across emitters create false peaks.

When timing is not enough, pulses are clustered in a joint space of RF, pulse width, AOA, and related measurements, using k-means or density-based methods [4, 5]. Deinterleaving is then a clustering problem in a chosen feature space. Cluster count, metric, and scaling have to be set by hand, which is awkward when the number of emitters is unknown.

Classical architectures treat train formation as an early, discrete step. Later stages are expected to see one dominant hypothesis per emitter, or a short ranked list, rather than the interleaved mixture [2].

2.2.2 Mode analysis: grouping pulses by operating mode

Mode analysis asks a different question of the same pulses: which transmit schedule is active. In the classical chain this step runs after deinterleaving, on each recovered train rather than on the mixture [2]. The train is matched to a mode library of nominal RF bands, PRI/PRF families, stagger laws, and scan programs.

A correctly deinterleaved train can still contain several mode segments, because one emitter switches modes as its task changes. A mode match is not an emitter identity either, because several emitters can implement the same functional mode. When the library is incomplete, analysts group trains, or segments of trains, by similar behavior and name those groups later.

Equipment identification is a further lookup. Aggregated train statistics are compared with an emitter library to propose a radar type. That step names hardware against a catalogue. It is not required in order to form the emitter and mode partitions, and it is not the subject of this thesis.

Errors in deinterleaving enter mode analysis as corrupted trains. Nothing in this chain produces both groupings from the mixed stream at once.

2.3 Machine Learning Fundamentals

ML replaces hand-coded decision rules with parametric models fitted to data. A model is a function $f_{\boldsymbol{\theta}}\colon \mathcal{X} \to \mathcal{Y}$ from an input space $\mathcal{X}$ to a task-specific output space $\mathcal{Y}$. Learning searches over parameters $\boldsymbol{\theta}$ so that $f_{\boldsymbol{\theta}}$ matches the training examples under a chosen loss. Problems differ in the supervision available, the structure of $\mathcal{Y}$, and that loss.

The usual training objective is empirical risk minimization. Given a dataset $\mathcal{D} = \{(\boldsymbol{x}_i, y_i)\}_{i=1}^{N}$ drawn from an unknown distribution and a pointwise loss ℓ, the learner solves

$$\hat{\boldsymbol{\theta}} = \arg\min_{\boldsymbol{\theta}} \; \frac{1}{N} \sum_{i=1}^{N} \ell\Big(f_{\boldsymbol{\theta}}(\boldsymbol{x}_i),\, y_i\Big) + \Omega(\boldsymbol{\theta}), \tag{2.5}$$

where Ω is an optional regularizer that trades fit to the training data against complexity of the fitted model. Generalization is measured on held-out data. The training procedure as a whole (data, model class, loss, optimizer, regularization) is judged by that out-of-sample behaviour, not by the training fit.

Learning paradigms

ML problems are grouped by the supervision available at training time. In supervised learning, each $\boldsymbol{x}_i$ has a label y_i, categorical for classification or real-valued for regression, and ℓ penalizes disagreement with that label. In unsupervised learning there are no labels. The aim is to expose structure in the input distribution, for example by density estimation, dimensionality reduction, or clustering. Section 2.4 treats clustering because this thesis uses it at inference to turn embeddings into emitter and mode partitions. Self-supervised learning sits between the two: it builds targets from the data itself (masked tokens, contrasted views, shuffled order) so an encoder can be trained without manual annotation. The remainder of this thesis is supervised. Training recordings carry per-pulse emitter and operating-mode labels (Sections 4.1 and 4.2), and the objective uses both streams. Unsupervised and self-supervised ideas still matter here because the clustering tools in Section 2.4 and the contrastive machinery in Section 2.6 come from those traditions.

Representation learning

Much of DL trains an encoder that maps observations to a lower-dimensional vector whose geometry is shaped by the loss. Later steps (classification, clustering, retrieval, sequence prediction) then use the embeddings $\boldsymbol{z} = f_{\boldsymbol{\theta}}(\boldsymbol{x}) \in \mathbb{R}^d$ rather than the raw coordinates. Invariances and similarities need only be learned once, in $f_{\boldsymbol{\theta}}$; simple operations on $\boldsymbol{z}$ often suffice after that. Sections 2.6 and 2.7 develop the losses and the encoder. The method chapter (Chapter 4) applies this to PDW streams, with embeddings meant to expose emitter and mode structure.

Applications in radar deinterleaving and mode classification

Deinterleaving and mode classification are the two tasks this thesis addresses. For a long time both were handled by temporal rules and hand-crafted statistics on PDW parameters (Section 2.2). As emitters became more agile and environments more crowded, the field moved first to shallow clustering (k-means and density-based methods over RF, AOA, and pulse width) and then to deep sequence models that learn pulse-level features from data [4]. Recurrent and attention-based models treat deinterleaving as sequence labelling: the per-pulse output is an emitter. Transformer encoders can relate pulses that are not neighbours in time, which is useful for jittered, staggered, or grouped PRIs [4, 6].

Mode classification followed the same path. Classical ESM systems match deinterleaved trains to waveform templates in a mode library [2]. Learning-based work treats the task as supervised classification over known modes, or as unsupervised grouping of trains when labels are scarce [4]. This thesis uses per-pulse mode labels from a global catalogue at training time, but the loss is supervised contrastive rather than a closed-catalogue softmax. The emitter term is the same kind of contrastive signal, with labels that are local to each recording. Inference on both branches is per-window clustering. Sections 2.4, 2.6 and 2.7 supply the tools the method chapter uses.

2.4 Clustering Methods

Clustering partitions a finite set of points $\{\boldsymbol{z}_i\}_{i=1}^{N} \subset \mathbb{R}^d$ into groups so that points inside a group are more similar to one another than to points in other groups. In this thesis the points are pulse embeddings and the groups are emitters within a window or operating modes. The clustering step runs only at inference, after the encoder has been trained (Section 4.3). Classical algorithms differ in how similarity is defined, whether the number of clusters must be known in advance, and how they treat outliers.

Centroid-based methods such as k-means assign every point to the nearest of k means and update those means until the partition stabilizes. They are fast and well understood, but they assume roughly spherical, equally scaled clusters, require a prescribed k, and force every point into some cluster. Mixture models such as a GMM replace the hard assignment with soft responsibilities under a parametric density. They still usually fix the number of components, and they remain sensitive to initialization and to elongated or overlapping structure. When the number of concurrent emitters or modes is unknown and the embedding space may contain noise or irregular densities, density-based methods fit better.

DBSCAN

DBSCAN recovers clusters from local density rather than from a fixed set of centres [5]. Two hyperparameters define the density criterion: a neighbourhood radius $\varepsilon > 0$ and a

minimum neighbourhood size minPts $\in \mathbb{N}$. A point is a core point if at least minPts points (including itself) lie inside its Euclidean ball of radius ε. Points that are not cores but fall inside some core's ball are border points. The remainder are labelled noise. Clusters grow by connecting core points that fall in each other's ε-neighbourhoods and attaching their border points. The number of clusters is then an output of the run, not an input.

This matches the inference setting of Equation (4.14): within each window the number of active emitters (and, separately, of modes) is not known in advance, and sparse outliers can be left unassigned instead of distorting the partition. The main practical cost is sensitivity to ε. A single global radius assumes that all clusters share a comparable density scale. The method chapter uses DBSCAN with frozen $(\varepsilon, \text{minPts})$ on each branch embedding.

Applied to raw high-dimensional features, density thresholds degrade because neighbourhoods become less informative as d grows. Deep clustering therefore first learns a lower-dimensional embedding in which the desired partitions are easier to separate, then runs a classical operator such as k-means or DBSCAN in that space [7]. How a recovered partition is scored against a reference is defined in Section 2.5. Section 2.6 reviews the representation-learning side of that pipeline.

2.5 Cluster Agreement Metrics

A clustering algorithm returns a labelling of N points. The integers used as cluster names are arbitrary: two labellings that group the same points together are the same partition. External evaluation therefore compares a predicted partition $\hat{Y}$ with a reference partition Y in a way that does not depend on those names.

All scores below are computed from the contingency table

$$n_{ij} = \sum_{n=1}^{N} \mathbf{1}[y_n = i,\, \hat{y}_n = j], \tag{2.6}$$

with row sums $a_i = \sum_j n_{ij}$ and column sums $b_j = \sum_i n_{ij}$ equal to the sizes of the reference and predicted clusters.

Adjusted Rand index

Each unordered pair of points is either together or apart in a given partition. A cluster of size a_i contributes $\binom{a_i}{2}$ co-clustered pairs, so

$$S_{Y\hat{Y}} = \sum_{i,j} \binom{n_{ij}}{2}, \qquad S_Y = \sum_i \binom{a_i}{2}, \qquad S_{\hat{Y}} = \sum_j \binom{b_j}{2} \tag{2.7}$$

count the pairs that share a cluster in both labellings, in Y, and in $\hat{Y}$, out of $\binom{N}{2}$ pairs in total.

The Rand index is the fraction of pairs on which Y and $\hat{Y}$ agree: together in both, or apart in both [8],

$$R(Y,\hat{Y}) = 1 - \frac{S_Y + S_{\hat{Y}} - 2\,S_{Y\hat{Y}}}{\binom{N}{2}} \in [0,1]. \tag{2.8}$$

The numerator $S_Y + S_{\hat{Y}} - 2\,S_{Y\hat{Y}}$ is the number of pairs that are together in one partition and apart in the other.

When most points lie in different clusters, many pairs are apart in both labellings by chance, so R stays close to one even when the partitions disagree.

If the predicted labels are shuffled at random, keeping the cluster sizes, a pair is together in $\hat{Y}$ with probability $S_{\hat{Y}}/\binom{N}{2}$. The expected number of pairs that are also together in Y is therefore

$$\mathbb{E}[S_{Y\hat{Y}}] = \frac{S_Y\,S_{\hat{Y}}}{\binom{N}{2}}. \tag{2.9}$$

With the sizes fixed, R in Equation (2.8) moves only with $S_{Y\hat{Y}}$. Subtracting this chance overlap, and dividing by the overlap at a perfect match, $\frac{1}{2}(S_Y + S_{\hat{Y}})$, gives [8]

$$\mathrm{ARI} = \frac{S_{Y\hat{Y}} - \mathbb{E}[S_{Y\hat{Y}}]}{\frac{1}{2}\big(S_Y + S_{\hat{Y}}\big) - \mathbb{E}[S_{Y\hat{Y}}]}, \qquad \mathrm{ARI} \le 1. \tag{2.10}$$

Then $\mathrm{ARI} = 1$ at exact recovery up to relabelling, $\mathrm{ARI} \approx 0$ at chance, and negative values are worse than random.

Adjusted mutual information

Mutual information is the reduction in uncertainty about one labelling given the other [9],

$$\mathrm{MI}(Y;\hat{Y}) = H(Y) - H(Y \mid \hat{Y}) = H(\hat{Y}) - H(\hat{Y} \mid Y). \tag{2.11}$$

$H(Y \mid \hat{Y})$ is the uncertainty left in Y once $\hat{Y}$ is known, and likewise with the labels swapped. The quantity is zero when the labellings are independent, and equals $H(Y)$ or $H(\hat{Y})$ when one labelling determines the other.

The Shannon entropies of the cluster-size distributions are

$$H(Y) = -\sum_{i:\,a_i>0} \frac{a_i}{N} \log \frac{a_i}{N}, \qquad H(\hat{Y}) = -\sum_{j:\,b_j>0} \frac{b_j}{N} \log \frac{b_j}{N}. \tag{2.12}$$

The plug-in estimate of Equation (2.11) from Equation (2.6) is

$$\mathrm{MI}(Y;\hat{Y}) = \sum_{i,j:\,n_{ij}>0} \frac{n_{ij}}{N}\, \log \frac{N\,n_{ij}}{a_i b_j}, \tag{2.13}$$

with the convention $0 \log 0 = 0$.

The expectation of MI under the same permutation null grows with the number of clusters. The adjusted mutual information AMI subtracts that baseline and rescales by the average of the marginal entropies [9],

$$\mathrm{AMI}(Y; \hat{Y}) = \frac{\mathrm{MI}(Y; \hat{Y}) - \mathbb{E}[\mathrm{MI}]}{\frac{1}{2}\Big(H(Y) + H(\hat{Y})\Big) - \mathbb{E}[\mathrm{MI}]}, \tag{2.14}$$

when the denominator is positive; otherwise $\mathrm{AMI} \equiv 0$. The range matches ARI. AMI weights clusters more evenly than pair counting, so large groups dominate it less.

Homogeneity, completeness, and V-measure

Homogeneity is high when each predicted cluster contains points from only one reference class. Completeness is high when each reference class is collected into a single predicted cluster. Splitting a class across many predicted clusters preserves homogeneity and lowers completeness; merging several classes into one predicted cluster does the reverse.

Expanding the conditional entropies of Equation (2.11) gives

$$H(Y \mid \hat{Y}) = -\sum_{i,j:\, n_{ij}>0} \frac{n_{ij}}{N} \log \frac{n_{ij}}{b_j}, \qquad H(\hat{Y} \mid Y) = -\sum_{i,j:\, n_{ij}>0} \frac{n_{ij}}{N} \log \frac{n_{ij}}{a_i}. \tag{2.15}$$

Homogeneity and completeness are then [10]

$$h = 1 - \frac{H(Y \mid \hat{Y})}{H(Y)}, \qquad c = 1 - \frac{H(\hat{Y} \mid Y)}{H(\hat{Y})}, \tag{2.16}$$

with $h = 1$ if $H(Y) = 0$ and $c = 1$ if $H(\hat{Y}) = 0$. The V-measure is their harmonic mean,

$$\mathrm{V}(Y, \hat{Y}) = \frac{2hc}{h + c}, \qquad h + c > 0, \tag{2.17}$$

and $\mathrm{V} = 0$ if $h = c = 0$. The harmonic mean is small if either factor is small, so a high V-measure requires both.

The three families return one at exact recovery and about zero at chance. Between those extremes, ARI follows pair agreement, AMI follows shared information, and homogeneity versus completeness separates splitting from merging.

2.6 Deep Representation Learning

Many DL pipelines replace hand-crafted features with a parametric map that is trained end-to-end from data. The central object is an *encoder* $f_{\boldsymbol{\theta}} \colon \mathcal{X} \to \mathbb{R}^d$, implemented in practice as a stack of nonlinear layers (for example, convolutions over a grid, recurrence over time, or attention over a sequence) followed by a pooling or readout head that

yields a fixed-dimensional embedding $\boldsymbol{z} = f_{\boldsymbol{\theta}}(\boldsymbol{x})$. The design question is which training signal makes $\boldsymbol{z}$ useful for the downstream task when labels are scarce or absent.

In supervised learning, $f_{\boldsymbol{\theta}}$ is trained together with a classifier so that $\boldsymbol{z}$ separates classes under a cross-entropy or margin loss. When cluster structure is the target but assignments are unknown, the same encoder can instead be trained with objectives that encourage smoothness, invariance to nuisance transformations, or agreement with a clustering hypothesis in embedding space. *Self-supervised* learning occupies an intermediate regime: the model predicts surrogate targets constructed from the input itself (for example, masked tokens, relative positions, or transformed views), so that $f_{\boldsymbol{\theta}}$ learns semantics without human labels [11].

Contrastive objectives implement instance discrimination by treating two augmentations of the same example as a positive pair and other examples in a minibatch (or a memory bank) as negatives. Let $\boldsymbol{z}_i$ and $\boldsymbol{z}_i^+$ denote normalized embeddings of two views of sample i, let $\mathrm{sim}(\cdot,\cdot)$ denote cosine similarity, and let $\tau > 0$ be a temperature. A standard *InfoNCE*-style loss takes the form

$$\mathcal{L}_{\mathrm{NCE}}(i) = -\log \frac{\exp\Big(\mathrm{sim}(\boldsymbol{z}_i, \boldsymbol{z}_i^+)/\tau\Big)}{\exp\Big(\mathrm{sim}(\boldsymbol{z}_i, \boldsymbol{z}_i^+)/\tau\Big) + \sum_{j \neq i} \exp\Big(\mathrm{sim}(\boldsymbol{z}_i, \boldsymbol{z}_j^+)/\tau\Big)}, \tag{2.18}$$

which lower-bounds mutual information between representations of different views under suitable generative assumptions [12]. Large-scale implementations such as SimCLR combine strong augmentations with wide embeddings and many negatives drawn from the same minibatch [11]. Equation (2.18) should be read as a template: the positive set can be enlarged, negatives can be drawn from a queue, and the similarity can be implemented with a learned projection head without changing the underlying contrastive principle.

Pretext tasks offer an alternative route: the network solves an auxiliary problem (predicting rotations, solving jigsaw permutations, inpainting masked regions) whose solution is incidental, but the representation $\boldsymbol{z}$ becomes predictive of semantically stable factors. Such objectives can be combined with clustering when the end goal is partition discovery rather than linear probe accuracy on a fixed label set. Section 3.3 surveys deep embedded clustering and prototype-based alternatives that couple encoder training with discrete structure, together with the gap relative to nested emitter–mode organization in ELINT streams.

Sequence encoders based on self-attention, including the transformer blocks reviewed in Section 2.7, instantiate $f_{\boldsymbol{\theta}}$ when $\boldsymbol{x}$ is an ordered set of tokens (for example, a window of PDWs). The Method chapter builds on this stack: a contrastive or clustering-friendly loss shapes the embedding space, while the attention-based encoder models temporal context within each window.

2.7 The Transformer Encoder

The transformer architecture replaces recurrence with a stack of layers built around *self-attention*, so that every position in a sequence can directly attend to every other position in parallel [13]. One understanding of this is a weighted fully connected graph. Encoder-only stacks, as in BERT [14], map an input sequence to contextualised token representations of the same length and are a natural choice when the downstream task is representation learning or clustering over tokens rather than autoregressive generation.

Scaled dot-product attention. Attention maps three matrices of *queries* $\boldsymbol{Q}$, *keys* $\boldsymbol{K}$, and *values* $\boldsymbol{V}$ to an output matrix of the same leading dimension as $\boldsymbol{Q}$. Each row of $\boldsymbol{Q}$ scores all rows of $\boldsymbol{K}$ through inner products; the scores are scaled, normalised with a row-wise softmax, and used to form a convex combination of the rows of $\boldsymbol{V}$. With d_k the dimension of each query and key vector (column width of $\boldsymbol{Q}$ and $\boldsymbol{K}$ after the usual transpose convention), the mapping is

$$\text{Attention}(\boldsymbol{Q}, \boldsymbol{K}, \boldsymbol{V}) = \text{softmax}\left(\frac{\boldsymbol{Q}\boldsymbol{K}^{\mathsf{T}}}{\sqrt{d_k}}\right)\boldsymbol{V}. \tag{2.19}$$

The scaling by $\sqrt{d_k}$ keeps the inner products from growing too large in magnitude as d_k increases, which would otherwise drive the softmax into near one-hot rows and yield vanishing gradients [13]. Equation (2.19) is reused later as the core attention primitive inside the encoder.

Self-attention and MHA. In *self-attention*, $\boldsymbol{Q}$, $\boldsymbol{K}$, and $\boldsymbol{V}$ are all linear projections of the same input sequence layout, so a token's representation is updated by aggregating information from the full sequence. MHA runs h attention mechanisms in parallel on different learned subspaces, concatenates the head outputs, and applies one more linear map to restore the model width [13]. The heads specialise to different relational patterns while sharing the same residual and normalisation wrapper as a single conceptual operator.

Positional information. Self-attention is permutation-equivariant in its inputs: reordering the rows of $\boldsymbol{Q}$, $\boldsymbol{K}$, and $\boldsymbol{V}$ in the same way reorders the output rows accordingly, but does not otherwise encode order. Transformer encoders therefore inject a positional signal. The original architecture adds fixed sinusoidal features or learned embeddings to each token vector before the first layer [13, 14]. Those encodings are absolute. They tag a token with its index, not with its distance to other tokens.

Rotary positional encoding. Su et al. [15] introduce RoPE. The query and key of each token are rotated by a phase that depends on a scalar coordinate p, leaving the token embeddings unchanged. In the original formulation p_i is the token index

i. Write $\boldsymbol{q}_i = \boldsymbol{W}_q \boldsymbol{h}_i$ and $\boldsymbol{k}_i = \boldsymbol{W}_k \boldsymbol{h}_i$ for the usual per-head projections of a token representation $\boldsymbol{h}_i$. The rotated vectors are functions $f_q(\boldsymbol{h}_i, p_i)$ and $f_k(\boldsymbol{h}_j, p_j)$ of the hidden representation and the coordinate. Their inner product is required to depend on the two coordinates only through their difference,

$$\Big\langle f_q(\boldsymbol{h}_i, p_i),\, f_k(\boldsymbol{h}_j, p_j) \Big\rangle = g(\boldsymbol{h}_i, \boldsymbol{h}_j, p_j - p_i). \tag{2.20}$$

Assume d_k is even. The d_k coordinates of a head are grouped into $d_k/2$ two-dimensional planes. Plane m is rotated by the angle $p\omega_m$, where $\omega_m = b^{-2(m-1)/d_k}$ and $b = 10^4$ is a fixed base. Fast-rotating planes resolve small gaps in p. Slow-rotating planes resolve large ones. The same geometric progression of wavelengths appears in the sinusoidal encoding of Vaswani et al. [13]. Each plane uses the rotation

$$\boldsymbol{\Omega}(\phi) = \begin{bmatrix} \cos\phi & -\sin\phi \\ \sin\phi & \cos\phi \end{bmatrix}, \tag{2.21}$$

and the head-wide map is the block-diagonal matrix

$$\boldsymbol{R}(p) = \operatorname{diag}\Big(\boldsymbol{\Omega}(p\omega_1),\, \boldsymbol{\Omega}(p\omega_2),\, \ldots,\, \boldsymbol{\Omega}(p\omega_{d_k/2})\Big). \tag{2.22}$$

Applying $\boldsymbol{R}(p)$ after the projections gives

$$\tilde{\boldsymbol{q}}_i = \boldsymbol{R}(p_i)\, \boldsymbol{q}_i, \qquad \tilde{\boldsymbol{k}}_j = \boldsymbol{R}(p_j)\, \boldsymbol{k}_j. \tag{2.23}$$

Planar rotations compose by subtracting angles, $\boldsymbol{\Omega}(\phi)^\top \boldsymbol{\Omega}(\psi) = \boldsymbol{\Omega}(\psi - \phi)$. Block-wise this is $\boldsymbol{R}(p_i)^\top \boldsymbol{R}(p_j) = \boldsymbol{R}(p_j - p_i)$, so

$$\Big\langle \tilde{\boldsymbol{q}}_i,\, \tilde{\boldsymbol{k}}_j \Big\rangle = \boldsymbol{q}_i^\top\, \boldsymbol{R}(p_j - p_i)\, \boldsymbol{k}_j. \tag{2.24}$$

Each token is rotated by its own coordinate, yet the logit sees only the difference. Setting $\boldsymbol{R}(p) = \boldsymbol{I}$ recovers the content-only inner product. Figure 2.3 shows the geometry on a single plane.

The construction applies to any real scalar p, not only to integer indices. When the token features already carry timing, a separate positional encoding is sometimes omitted altogether. The method chapter returns to this for PDW windows: a baseline that omits a separate encoding, and a rotary encoding that uses physical coordinates in place of token indices.

Encoder layer. A standard transformer *encoder layer* alternates two sub-layers: MHA followed by a position-wise FFN applied independently at every position. Each sub-layer uses a residual connection and layer normalisation [13]. The FFN is typically a two-layer MLP with a wide inner dimension and a nonlinear activation, acting as the per-token nonlinearity while MHA handles cross-token mixing. Stacking N such

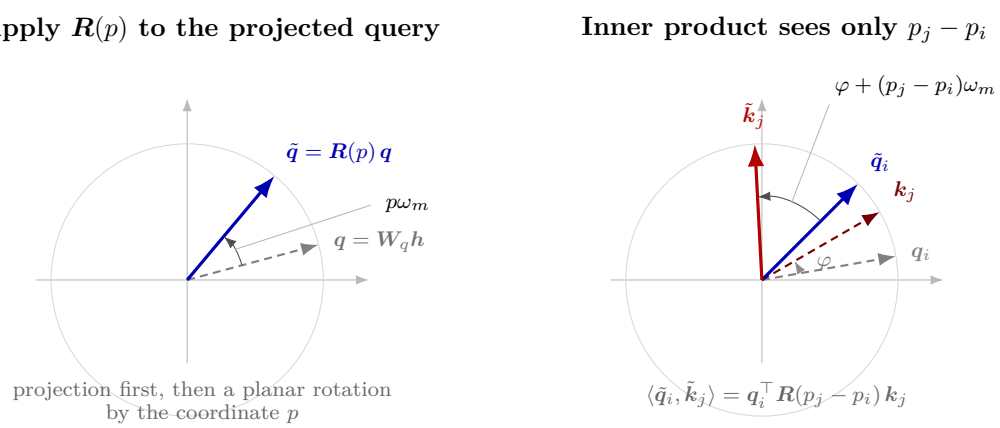


Figure 2.3. Geometry of RoPE on one two-dimensional plane of a head. Left: the projected query $\boldsymbol{q} = \boldsymbol{W}_q\boldsymbol{h}$ (dashed) is rotated by $\boldsymbol{R}(p)$ from Equation (2.22), which adds the phase $p\omega_m$. Right: the angle between the rotated query and key equals the content angle φ plus the relative phase $(p_j - p_i)\omega_m$. The inner product depends on the coordinate difference alone (Equation (2.24)).

layers yields a deep encoder that maps an input sequence $(\boldsymbol{x}_1, \ldots, \boldsymbol{x}_L)$ to contextualised representations $(\boldsymbol{h}_1^{(N)}, \ldots, \boldsymbol{h}_L^{(N)})$, which downstream heads can pool or read out for sequence-level or token-level predictions.

Computational cost. In the usual dense implementation, each self-attention block materialises an $L \times L$ matrix of token–token scores before the softmax, so time and activation memory for that block scale as $\mathcal{O}(L^2)$ in the sequence length L at fixed embedding width, up to constants from the number of heads and projection ranks [13]. Repeating the block across depth multiplies that cost by the number of layers, which motivates hard caps on L, sparse or low-rank attention variants, or chunked processing when inputs are long; the method chapter states the windowing choice adopted here.

CHAPTER 3

Related Work

This chapter reviews work that the method in Chapter 4 builds on or differs from. The first two sections cover radar-specific pipelines: classical sorting and identification, then learned classifiers and embedding models for PDWs. The remaining sections cover the learning ingredients used later—deep clustering, transformers on irregular sequences, and clustering at more than one granularity. Each section ends with the limitation that the next chapter addresses.

3.1 Classical Radar Emitter Identification

Operational ELINT systems recover emitter identity from measured PDWs by combining pulse sorting with library lookup [2, 3]. After detection, pulses are grouped into trains and each train is matched against stored templates of carrier frequency, pulse width, PRI family, and scan behavior. The grouping step is deinterleaving; the matching step is identification. Both historically depend on hand-specified features and on regularity that agile radars no longer guarantee.

The oldest deinterleavers exploit repetition in TOA. The cumulative difference histogram (CDIF) accumulates TOA differences at increasing pulse lags and treats histogram peaks as candidate PRIs, which a subsequent search then validates as pulse sequences [16]. The sequential difference histogram (SDIF) keeps only the current difference level, with a derived detection threshold, which avoids accumulating every lag while remaining a TOA-only method [17]. These algorithms are reliable when each emitter keeps an approximately constant PRI and when missing pulses and cross-emitter differences do not create false peaks. They degrade under stagger, jitter, and dense mixing, which is the setting of the recordings in Section 4.1.

When timing alone is unstable, sorting moves into a joint space of RF, pulse width, and AOA. Centroid methods such as k-means and density clustering such as DBSCAN then replace histogram peak finding [4, 5]. Density clustering has the operational advantage that the number of emitters need not be specified in advance. Its accuracy still depends on the metric, the scaling of each coordinate, and a neighbourhood radius that is hard to set when cluster densities differ. Offline direction clustering of overlapping pulses is one concrete instance of this approach [18]. The method chapter uses DBSCAN only after a learned embedding; applying it to raw window features is retained as a baseline (Section 4.8).

Once trains are formed, identification is typically template matching against a mode or equipment library [2]. Statistical fingerprints—histograms of RF and PRI, dwell

statistics, scan-period estimates—summarize each train. The match assumes that the library contains the emitter and that deinterleaving errors have not corrupted the fingerprint. Novel emitters and modes that are absent from the catalogue fall outside this procedure by construction.

Classical pipelines treat deinterleaving and mode analysis as successive stages, each with its own hand-engineered features. They do not learn a representation in which both partitions are visible, and they have no mechanism for emitters that appear only at inference. Those two requirements—recording-local emitter grouping and a mode grouping that is not a closed-catalogue softmax—motivate the learned encoder of Chapter 4.

3.2 Deep Learning for Radar Signal Analysis

Learned models replace histogram rules with a parametric map from PDW sequences or time-frequency images to labels. Most published systems are supervised classifiers: they assume a global catalogue of emitter types, or of intra-pulse modulations, and they train a CNN or RNN with cross-entropy [4]. Attribute-specific recurrent networks on PDW streams are a representative sequence model in this family [19]. They capture temporal dependence that a bag of pulse parameters misses, but the output is still a class index in a fixed label set.

A closer relative is supervised contrastive training for emitter classification. Jonsson [20] trains an encoder so that samples with the same emitter label lie close in embedding space, using synthetic scenarios from the same simulator lineage as Section 4.2. The loss belongs to the family used in Section 4.5, but the task is still closed-set identification: emitter identifiers are treated as globally comparable classes, and inference is classification rather than clustering. Recurrent ensembles for emitter classification under dataset drift make the same closed-set assumption and treat out-of-distribution detection as a separate module [21].

Deinterleaving as clustering, with an unknown number of emitters, is a different problem. Gunn et al. [6] train a transformer encoder with a triplet loss so that, within one interleaved stream, pulses from the same emitter are close and pulses from different emitters are far. At inference they cluster the embeddings with hierarchical density clustering (HDBSCAN) [22, 23]. They do not assume a known emitter count, they evaluate with AMI, ARI, and V-measure, and they omit index-based positional encodings because TOA already orders the pulses. That pipeline is the closest published counterpart to Section 4.3.

Three differences remain. First, Gunn et al. produce a single embedding and a single partition—emitter identity—and treat operating mode as a later analysis that becomes easier after deinterleaving. Second, they train with a triplet hinge, which uses one positive and one negative per anchor; the method in Section 4.5 uses the supervised contrastive kernel of Khosla et al. [24], which averages over all positives of an anchor

in the mini-batch. Third, their transformer uses vanilla dot-product attention. This thesis additionally evaluates a pairwise physical bias and a rotary encoding of time and azimuth (Section 4.4).

Deep models for radar pulses already exist. What they still omit is the joint problem of this thesis: two embeddings whose positive sets have different scopes—recording-local emitters and globally shared modes—both decoded by clustering rather than by a catalogue softmax.

3.3 Deep Clustering and Contrastive Representation Learning

Classical clustering assumes that either the raw features or a fixed embedding already reveal the desired partition. Deep clustering drops that split: the encoder and the discrete structure are learned together.

DEC places trainable centroids in embedding space and alternates soft assignment with a Kullback–Leibler objective that sharpens confident assignments toward an auxiliary target [7]. IDEC adds a reconstruction term so that the clustering loss does not destroy local geometry [25]. DeepCluster instead runs k-means on the current features and treats the assignments as pseudo-labels for a discriminative update [26]. In all three, the number of clusters is a design choice, and the only partition being learned is a single flat grouping.

Contrastive methods replace explicit centroids with pairwise attraction and repulsion. InfoNCE and SimCLR pull two views of the same instance together and push other batch members apart [11, 12]. SwAV predicts a small codebook of prototypes from one view and uses the assignment of the other view as the target, which avoids comparing every pair while still preventing collapse [27]. Contrastive clustering applies the same idea in both the instance dimension and the cluster dimension of a feature matrix [28]. These objectives are unsupervised: positives are defined by augmentation, not by labels.

When labels are available, the positive set can be defined by class identity rather than by instance identity. Supervised contrastive learning does exactly that: every other sample with the same class is a positive, and the loss is an InfoNCE average over those positives [24]. It recovers the triplet loss of FaceNet as a special case that uses a single positive and a hard negative [29]. The method of Section 4.5 uses this kernel twice, with different rules for who counts as a positive.

Two properties of this literature matter for the method. First, training and assignment are usually coupled: DEC, DeepCluster, and SwAV produce cluster indices as part of the loss. This thesis decouples them. The contrastive terms shape the embedding; DBSCAN is applied only at inference and contributes no gradient (Equation (4.14)). Second, almost every method learns one partition. A single contrastive term cannot encode both a recording-local emitter grouping and a globally comparable mode group-

ing, because those two relations contradict each other in one embedding: two pulses from different emitters can share a mode, and two pulses from one emitter can differ in mode. That conflict is why the encoder splits into two branches after a shared trunk.

3.4 Transformers for Time Series and Sets

The transformer encoder maps a sequence of tokens to contextualised representations by unmasked self-attention [13, 14]. Every pulse in a window can attend to every other pulse, which matches the structure of interleaved PDWs: the next pulse of the same emitter is generally not the next token. The cost is quadratic in the window length, which is why Section 4.2.2 processes recordings in fixed-length windows.

Self-attention is permutation-equivariant. Without a positional signal it treats the window as a set, which is the setting studied by the Set Transformer [30]. Absolute sinusoidal encodings and learned position embeddings restore order for language, where the index of a token is a meaningful coordinate [13]. For PDWs that coordinate is the wrong one. Consecutive pulses are not equally spaced in time, so an ordinal position misrepresents TOA gaps. Multivariate time-series transformers typically assume regular sampling and add a time-step index in the same way [31]. Gunn et al. [6] therefore omit positional encodings and rely on TOA as an input feature. The baseline encoder of Section 4.3 makes the same choice.

Relative encodings offer a middle path. Shaw et al. [32] add a learned embedding of the discrete offset between token indices to the attention logits. Graphormer generalises that idea to pairwise biases derived from graph distances [33]. The physical attention bias in Section 4.4.1 is the same mechanism with continuous pulse parameters: the absolute TOA difference and the circular azimuth distance, each scaled by a learned per-head coefficient.

Su et al. [15] rotate the queries and keys (Section 2.7). The inner product then depends on content and on the coordinate difference. Vision models split the rotation planes across spatial axes [34]. The method replaces token indices with physical coordinates, reuses that split for cosine and sine of azimuth on the deinterleaving branch, and rotates the trunk and mode branch by TOA only (Section 4.4.2).

Existing positional schemes were designed for discrete indices, regular grids, or a single shared coordinate. A PDW window is an irregular point set with at least two physically distinct relations—time of arrival and arrival direction—and the two estimation tasks should not weight those relations equally. Whether injecting them as a logit bias, as a rotary phase, or not at all improves clustering is taken up in Chapter 5.

3.5 Joint and Multi-Task Clustering

When two tasks share structure, a common pattern is a shared trunk with task-specific heads, trained by a weighted sum of losses [35]. Image models often attach two softmax

heads to one backbone, for example object class and attribute. That layout assumes both label sets are globally consistent catalogues. It does not by itself produce clusterable embeddings, and it cannot represent emitter identifiers that are comparable only within a recording.

Hierarchical clustering learns several partitions, but they are nested levels of one relation, not two different relations. Yang et al. [36] alternate agglomerative merges with representation updates under a triplet loss, so coarser groups are formed from finer ones of the same kind. A hierarchy of that form would be appropriate if modes were subtypes of emitters, or emitters subtypes of modes. They are not. One emitter uses several modes, and one mode appears on several emitters (Section 4.1). The two partitions also do not share a label scope: emitter symbols are recording-local, mode symbols are global.

A method that treats the two as softmax heads on a shared backbone therefore asks the wrong questions at inference. A method that learns a single embedding must compromise between two incompatible similarity relations: pulses that should be close as modes may need to be far as emitters, and conversely. The dual-branch encoder of Section 4.3 is the direct response to that conflict. The two supervised contrastive terms of Section 4.5 differ only in the scope of the positive set, and each branch is decoded by clustering rather than by a closed-catalogue classifier.

Chapter 4 therefore uses a transformer encoder on interleaved PDW streams, two contrastive terms with recording-local versus global positives, and independent density clustering of each branch at inference.

CHAPTER 4

Method

4.1 Problem Formulation

Observed input

The data consist of S recordings indexed by $s \in \{1, \ldots, S\}$, each an intercepted stream in which pulses from several emitters are interleaved in time. Recording s is observed as a finite, time-ordered sequence of PDWs

$$\boldsymbol{X}^{(s)} = (\boldsymbol{x}_1^{(s)}, \ldots, \boldsymbol{x}_{N_s}^{(s)}), \qquad \boldsymbol{x}_n^{(s)} \in \mathbb{R}^d,$$

where each $\boldsymbol{x}_n^{(s)}$ contains the measured parameters of one pulse (carrier frequency, amplitude, pulse width, time of arrival, and angles of arrival), and the length N_s varies between recordings. The encoder operates on fixed-length windows extracted from these recordings as described in Section 4.2.2. When a single recording is considered, the superscript is omitted.

Labels

Each pulse $n \in \{1, \ldots, N_s\}$ carries an emitter identity $e_n^{(s)} \in \{1, \ldots, K^{(s)}\}$ and an operating mode $m_n^{(s)} \in \{1, \ldots, M\}$. The emitter count $K^{(s)}$ may vary between recordings, and each emitter identifier is meaningful only within its recording. Mode labels, by contrast, are drawn from a finite catalogue of size M shared across all recordings. A recording generally contains only $M^{(s)} \leq M$ unique modes. The two labels describe different relations: one emitter may switch between several modes, and the same mode may occur on several emitters. The two labels of a pulse are collected into the ground-truth label pair

$$\boldsymbol{y}_n^{(s)} = \left(e_n^{(s)},\, m_n^{(s)}\right) \in \{1, \ldots, K^{(s)}\} \times \{1, \ldots, M\}, \tag{4.1}$$

which constitutes the complete per-pulse annotation of recording s.

For recording s, the coordinates of $\boldsymbol{y}_n^{(s)}$ induce emitter and mode cells on the pulse-index set,

$$\mathcal{E}_k^{(s)} = \{\, n : e_n^{(s)} = k \,\}, \qquad k \in \{1, \ldots, K^{(s)}\}, \tag{4.2}$$

$$\mathcal{M}_\ell^{(s)} = \{\, n : m_n^{(s)} = \ell \,\}, \qquad \ell \in \{1, \ldots, M\}. \tag{4.3}$$

The emitter cells are non-empty and partition $\{1, \ldots, N_s\}$. The mode cells are also

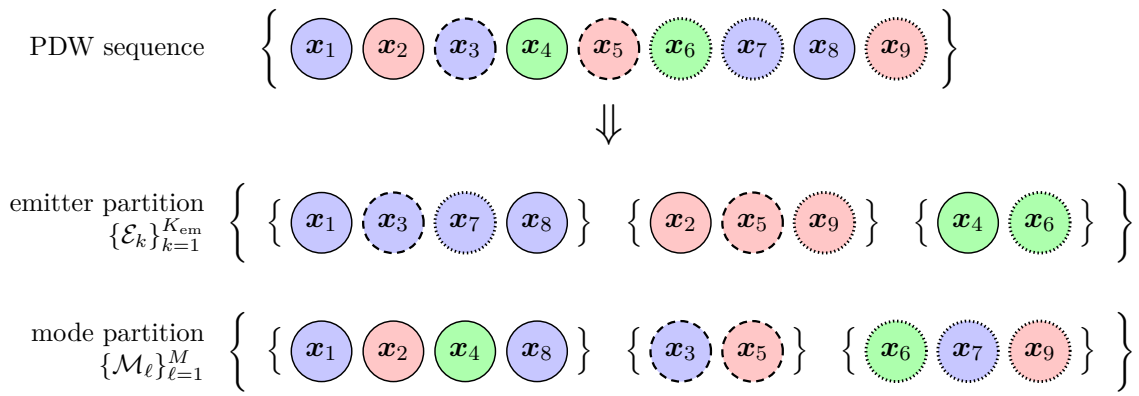


Figure 4.1. Each pulse $\boldsymbol{x}_n$ is assigned both an emitter label e_n (color) and a mode label m_n (border style).

disjoint, but $\mathcal{M}_\ell^{(s)}$ is empty when mode ℓ is absent from the recording; the non-empty mode cells form the corresponding partition. Recovering the emitter partition is the deinterleaving task, whereas recovering the mode partition groups pulses by the active mode for each PDW.

Estimation objective

At inference a time-ordered pulse sequence $\boldsymbol{X} = (\boldsymbol{x}_1, \ldots, \boldsymbol{x}_N)$ is observed and the label pairs of Equation (4.1) are unknown. The goal is to recover the emitter and mode partitions of that sequence, up to independent relabelling of the cluster indices in each coordinate. The estimated labels

$$\hat{\boldsymbol{y}}_n = \left(\hat{e}_n,\, \hat{m}_n\right) \in \{1, \ldots, \hat{K}\} \times \{1, \ldots, \hat{M}\}, \qquad n \in \{1, \ldots, N\}, \tag{4.4}$$

induce

$$\widehat{\mathcal{E}}_k = \{\, n : \hat{e}_n = k \,\}, \quad k \in \{1, \ldots, \hat{K}\}, \qquad \widehat{\mathcal{M}}_\ell = \{\, n : \hat{m}_n = \ell \,\}, \quad \ell \in \{1, \ldots, \hat{M}\}, \tag{4.5}$$

with $\hat{K}$ and $\hat{M}$ also unknown. Elementwise equality with the ground-truth symbols is not required; only the partitions matter, and they are scored with the permutation-invariant metrics of Section 4.7.

The method does not classify pulses against a fixed catalogue. It learns a map $f_{\boldsymbol{\theta}}$ that sends $\boldsymbol{X}$ to two sequences of pulse embeddings, one for emitters and one for modes. A clustering step applied to those embeddings produces Equation (4.4). All trainable weights of $f_{\boldsymbol{\theta}}$ are collected in the parameter vector $\boldsymbol{\theta}$, which is fitted by the loss of Section 4.5; the architecture is specified in Section 4.3.

4.2 Data Representation and Preprocessing

All training and evaluation data are synthetic because operational ELINT recordings are scarce and sensitive. A scenario simulator generates time-ordered PDW recordings with varying numbers of emitters and operating modes [20]. Each pulse contains the measured parameters and per-pulse labels defined in Section 4.1. The composition of the resulting corpus is reported in Section 5.2.

Preprocessing first standardizes carrier frequency, pulse width, and log-amplitude using statistics from the training split and converts the measured AOA into Euclidean direction components. Each recording is then divided into fixed-length windows. Absolute TOA is finally min–max mapped inside each window, so that the timing feature seen by the encoder depends only on pulses that co-occur in the same attention context, thereby reducing the risk of overfitting to scenario-specific timing characteristics and providing a more realistic representation of the information available at inference time. The encoder input is therefore the $d = 7$-dimensional vector of scaled carrier frequency, pulse width, log-amplitude, window-local TOA, and the three incidence-direction components.

No hand-crafted timing features are appended to this vector. Preliminary experiments augmented each pulse with lagged timing features, namely its instantaneous ΔTOA together with rolling means and standard deviations of ΔTOA over several look-back widths, to expose the local PRI structure explicitly; this configuration consistently degraded both emitter and mode clustering relative to the plain PDW input. A plausible explanation is that such look-back statistics summarize only the pulses preceding each position and mix contributions from all interleaved emitters, so under the pulse-count imbalance of the corpus (Section 5.2) they are dominated by the densest emitters and carry little signal for rare ones, whereas unmasked self-attention (Section 4.3) can relate each pulse to every other pulse in the window and recover inter-arrival structure per emitter. Letting the encoder learn timing structure implicitly also avoids committing to a fixed set of summary statistics, which keeps the representation free to adapt to interleaving patterns and PRI schedules that differ from those seen during training.

4.2.1 Per-feature scaling

Standardization statistics are estimated across all recordings in the training split and applied unchanged to validation and test data. Carrier frequency, pulse width, and log-amplitude are standardized per feature as

$$\tilde{g}_n = \frac{g_n - \mu_g}{\sigma_g}. \tag{4.6}$$

Here μ_g and σ_g are the corresponding training-set mean and standard deviation, and amplitude is log-transformed to reduce its dynamic range. The measured AOA consists of an azimuth θ_n^{az} and a latitude θ_n^{lat}, which together specify the incidence direction of the pulse on the unit sphere. Rather than feeding these angles to the encoder directly,

each pair is converted to the Euclidean unit direction vector

$$\boldsymbol{u}_n = \left(u_n^x,\, u_n^y,\, u_n^z\right)^\top = \left(\cos\theta_n^{\mathrm{lat}} \cos\theta_n^{\mathrm{az}},\, \cos\theta_n^{\mathrm{lat}} \sin\theta_n^{\mathrm{az}},\, \sin\theta_n^{\mathrm{lat}}\right)^\top, \tag{4.7}$$

whose components lie in $[-1, 1]$ and are mapped to $[0, 1]$ by the fixed affine rescaling

$$\tilde{u}_n^c = \frac{u_n^c + 1}{2}, \qquad c \in \{x, y, z\}. \tag{4.8}$$

Neither map depends on the training corpus.

The Cartesian representation avoids the wrap-around discontinuity of a rescaled angle, which places azimuths near 0 and 2π at opposite ends of the feature range although they describe nearly the same direction. Feeding incidence as continuous direction components therefore frees the encoder from having to learn that symmetry from the input features alone.

Corpus-wide standardization of frequency, pulse width, and amplitude preserves absolute ranges that may distinguish operating modes across recordings. Absolute TOA is left in physical units at this stage and is min–max mapped only after windowing (Equation (4.10)).

4.2.2 Windowing

Standard self-attention has time and memory complexity $\mathcal{O}(L^2)$ for a window of L pulses. Because a complete recording may contain millions of pulses, the encoder processes fixed-length windows rather than entire recordings. The value of L is reported with the experimental hyperparameters in Table 5.2.

Given a recording $\boldsymbol{X} = (\boldsymbol{x}_1, \ldots, \boldsymbol{x}_N)$ and stride δ, window w is

$$\boldsymbol{X}^{(w)} = \left(\boldsymbol{x}_{1+(w-1)\delta}, \ldots, \boldsymbol{x}_{L+(w-1)\delta}\right), \qquad w \in \left\{1, \ldots, \left\lfloor \frac{N-L}{\delta} \right\rfloor + 1\right\}. \tag{4.9}$$

The superscript (w) indexes windows within one recording, whereas (s) indexes recordings and is suppressed here. The number of windows therefore varies with the recording length. Any trailing segment shorter than L is dropped.

During training, choosing $\delta < L$ produces overlapping windows and therefore more training samples. A smaller stride improves coverage of sparse emitters and modes and reduces sensitivity to window boundaries, but it also repeats more of the same PDWs, increasing computation and correlation between samples. Advancing the window by a small stride resembles streaming inference because outputs can be refreshed as new pulses arrive, although it is not online learning because the model parameters remain fixed.

Validation and test use $\delta = L$, yielding non-overlapping windows that are processed entirely offline. Streaming inference and online model adaptation are left to future work (Section 7.3).

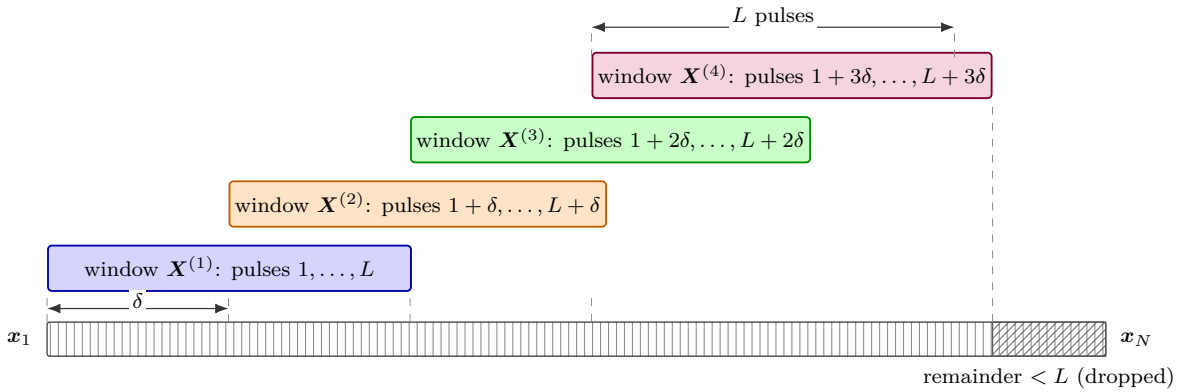


Figure 4.2. Sliding-window segmentation of a recording. Successive windows of length L advance by stride δ and overlap by $L-\delta$ pulses. A trailing segment shorter than L is dropped.

Each window retains the emitter label, mode label, and recording identifier associated with every pulse. The emitter and mode labels supervise their respective branches and provide evaluation targets. This makes the model and also evaluation dependent on the selected window length L, why a fixed L is used in this thesis.

The recording identifier is not an encoder input or prediction target; it is used to keep recording-local emitter identifiers distinct and to form scenario-aware mini-batches.

After windowing, absolute TOA is min–max mapped to $[0, 1]$ using only the L pulses inside the current window:

$$\tilde{t}_i^{(w)} = \frac{t_i^{(w)} - t_{\min}^{(w)}}{t_{\max}^{(w)} - t_{\min}^{(w)}}, \tag{4.10}$$

where $t_{\min}^{(w)}$ and $t_{\max}^{(w)}$ are the minimum and maximum TOA among the pulses of window w, and i indexes positions within that window. Per-window mapping keeps the TOA channel self-contained; when training uses $\delta < L$, the same physical pulse may receive different normalized values in overlapping windows, which is the intended behaviour for a window-local feature.

4.3 Transformer Encoder Architecture

The encoder $f_{\boldsymbol{\theta}}$ takes the form of a transformer-encoder backbone [13] with a shared trunk followed by two task-specific branches for mode clustering and deinterleaving. The overall layout is shown in Figure 4.3. The trunk produces a single contextualised representation of the input window, which the branches specialise into emitter-oriented and mode-oriented embeddings for the two estimation tasks defined in Section 4.1.

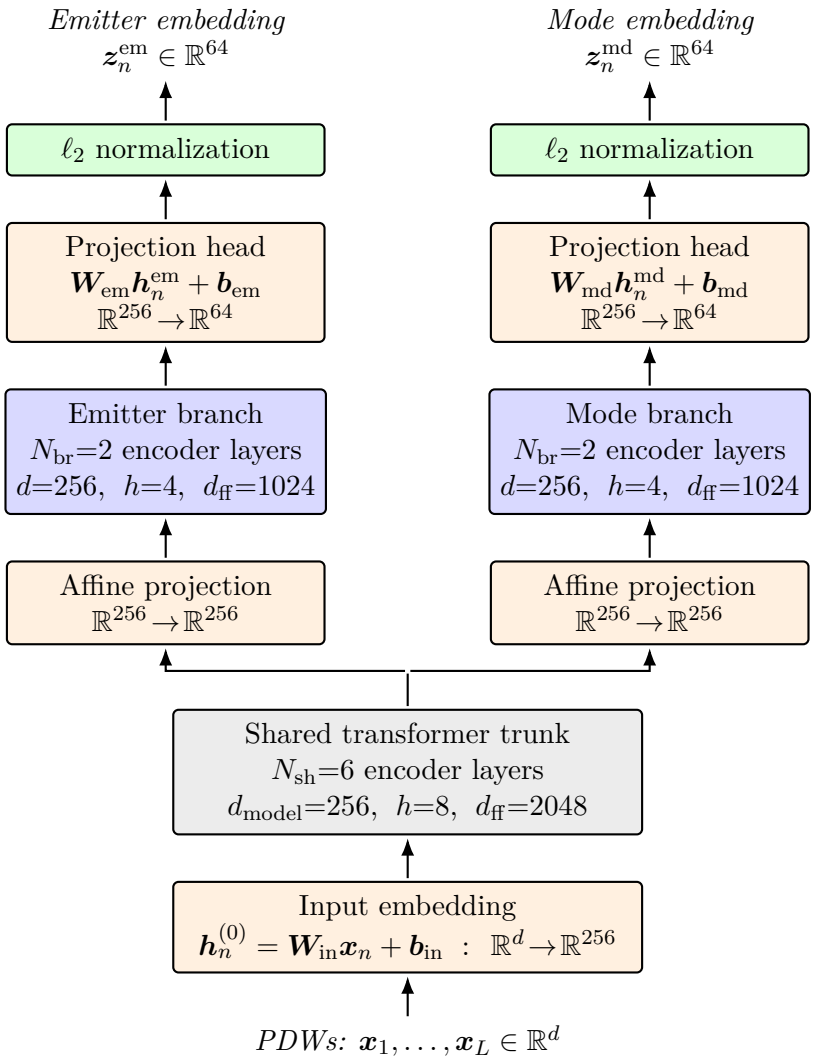


Figure 4.3. Architecture of the encoder $f_{\boldsymbol{\theta}}$. A sequence of PDWs is first lifted to $d_{\mathrm{model}} = 256$ by an affine input embedding, processed by a shared trunk of 6 transformer-encoder layers, and then split into two parallel branches, each with its own affine $256 \to 256$ projection and 2 encoder layers. Branch outputs are mapped by affine projection heads to the 64-dimensional emitter and mode embedding spaces and ℓ_2-normalized over the embedding dimension. The input embedding, branch projections, and projection heads are all affine maps of the same form.

Input embedding

Each pulse vector $\boldsymbol{x}_n \in \mathbb{R}^d$, built from the scaled PDWs of Section 4.2, is mapped to a token embedding through an affine projection

$$\boldsymbol{h}_n^{(0)} = \boldsymbol{W}_{\text{in}}\boldsymbol{x}_n + \boldsymbol{b}_{\text{in}}, \qquad \boldsymbol{h}_n^{(0)} \in \mathbb{R}^{d_{\text{model}}}, \tag{4.11}$$

where $\boldsymbol{W}_{\text{in}} \in \mathbb{R}^{d_{\text{model}} \times d}$ and d_{model} is the trunk width specified below. In the baseline configuration of the encoder, no positional encoding is added to $\boldsymbol{h}_n^{(0)}$. The temporal ordering of pulses is already carried by the TOA entry of $\boldsymbol{x}_n$; moreover, the index-based absolute positional encodings of the original architecture [13] are ill-suited to this input, since the TOA gap between consecutive pulses is not constant and an ordinal position therefore misrepresents the relative distances in time. Whether an explicit relative encoding of time and arrival direction nevertheless adds information beyond their presence as input features is revisited in Sections 4.4.1 and 4.4.2, where a pairwise attention bias and a rotary positional encoding acting on physical coordinate differences are evaluated as drop-in modifications of this architecture.

Shared trunk

The token sequence $(\boldsymbol{h}_1^{(0)}, \ldots, \boldsymbol{h}_L^{(0)})$ is processed by a stack of N_{sh} standard transformer-encoder layers, each comprising a MHA sub-layer and a position-wise FFN sub-layer with residual connections and layer normalisation around both. Self-attention is unmasked, so every pulse can attend to every other pulse in the window. The trunk operates with hidden dimension $d_{\text{model}} = 256$, $h_{\text{sh}} = 8$ attention heads (so that each head has dimension 32), and an inner FFN dimension of $d_{\text{ff,sh}} = 2048$; $N_{\text{sh}} = 6$ layers are used. Its output is a contextualised token sequence $\boldsymbol{H}^{\text{sh}} = (\boldsymbol{h}_1^{\text{sh}}, \ldots, \boldsymbol{h}_L^{\text{sh}})$ with $\boldsymbol{h}_n^{\text{sh}} \in \mathbb{R}^{256}$, in which information has been mixed across the full window.

Task-specific branches

The trunk output feeds two parallel branches that do not share parameters. Each branch begins with an affine projection of the same form as Equation (4.11), after which $N_{\text{br}} = 2$ transformer-encoder layers of the same form as the trunk are applied. The projection decouples the branch representation from the shared trunk output and, more generally, allows the branch width to differ from the trunk width; here the branches retain $d_{\text{br}} = 256$, so the projection maps $\mathbb{R}^{256} \to \mathbb{R}^{256}$. Within a branch, MHA uses $h_{\text{br}} = 4$ heads (per-head dimension 64) and the FFN inner dimension is $d_{\text{ff,br}} = 2048$. Write the resulting token sequences as $\boldsymbol{H}^{\text{em}} = (\boldsymbol{h}_1^{\text{em}}, \ldots, \boldsymbol{h}_L^{\text{em}})$ and $\boldsymbol{H}^{\text{md}} = (\boldsymbol{h}_1^{\text{md}}, \ldots, \boldsymbol{h}_L^{\text{md}})$ with $\boldsymbol{h}_n^{\text{em}}, \boldsymbol{h}_n^{\text{md}} \in \mathbb{R}^{256}$.

Projection heads

Each branch terminates in an affine projection head—of the same form as the input embedding Equation (4.11)—that maps every token to its respective embedding space, followed by ℓ_2 normalization over the embedding dimension,

$$\boldsymbol{z}_n^{\mathrm{em}} = \frac{\boldsymbol{W}_{\mathrm{em}}\boldsymbol{h}_n^{\mathrm{em}} + \boldsymbol{b}_{\mathrm{em}}}{\left\|\boldsymbol{W}_{\mathrm{em}}\boldsymbol{h}_n^{\mathrm{em}} + \boldsymbol{b}_{\mathrm{em}}\right\|_2}, \qquad \boldsymbol{z}_n^{\mathrm{md}} = \frac{\boldsymbol{W}_{\mathrm{md}}\boldsymbol{h}_n^{\mathrm{md}} + \boldsymbol{b}_{\mathrm{md}}}{\left\|\boldsymbol{W}_{\mathrm{md}}\boldsymbol{h}_n^{\mathrm{md}} + \boldsymbol{b}_{\mathrm{md}}\right\|_2}, \tag{4.12}$$

with $\boldsymbol{z}_n^{\mathrm{em}}, \boldsymbol{z}_n^{\mathrm{md}} \in \mathbb{R}^p$ for a common embedding dimension $p = 64$. Collecting the per-pulse outputs gives the two embedding sequences

$$\boldsymbol{Z}^{\mathrm{em}} = (\boldsymbol{z}_1^{\mathrm{em}}, \ldots, \boldsymbol{z}_L^{\mathrm{em}}), \qquad \boldsymbol{Z}^{\mathrm{md}} = (\boldsymbol{z}_1^{\mathrm{md}}, \ldots, \boldsymbol{z}_L^{\mathrm{md}}). \tag{4.13}$$

The normalization constrains both embedding sequences to the unit sphere in $\mathbb{R}^{64}$, so cluster structure is expressed through angular separation rather than vector magnitude; this is the geometry assumed by the cosine-similarity contrastive objective of Section 4.5.

From continuous embeddings to discrete partitions

The projection heads in Equation (4.12) produce continuous vectors, whereas the per-pulse outputs in Equation (4.4) are discrete. At inference time, each branch's L-token embedding sequence is therefore decoded independently and per window by a fixed clustering operator,

$$(\hat{e}_1, \ldots, \hat{e}_L) = \mathrm{DBSCAN}\Big(\{\boldsymbol{z}_n^{\mathrm{em}}\}_{n=1}^{L}\Big), \quad (\hat{m}_1, \ldots, \hat{m}_L) = \mathrm{DBSCAN}\Big(\{\boldsymbol{z}_n^{\mathrm{md}}\}_{n=1}^{L}\Big), \tag{4.14}$$

with cardinalities $\hat{K}, \hat{M} \in \mathbb{N}$ inferred from the data (Figure 4.4). DBSCAN [5] groups embeddings by local density at a Euclidean radius ε and minimum neighbourhood count minPts, assigning sparser points to a noise category. The radius ε is selected by a grid search over $\varepsilon \in \{0.1, 0.2, 0.3, 0.4, 0.5\}$ on the validation set, separately for each branch, using the clustering metrics of Section 4.7; the selected values are reported in Chapter 5. The neighbourhood count is fixed at minPts $= 5$, following its use in prior work on embedding-based deinterleaving [6]. With these settings the noise category remained marginal on the validation set—empty for most windows and almost never exceeding ten pulses per window—so no further treatment of noise-labelled pulses was deemed necessary. The clustering operator is applied only at inference, independently on each window and on each branch, and contributes no gradients. Cluster indices are therefore local to that window; the global mode labels of Section 4.1 enter only through the training loss. The trainable parameters $\boldsymbol{\theta}$ comprise the trunk, branch, and projection-head weights and are optimised by the loss of Section 4.5.

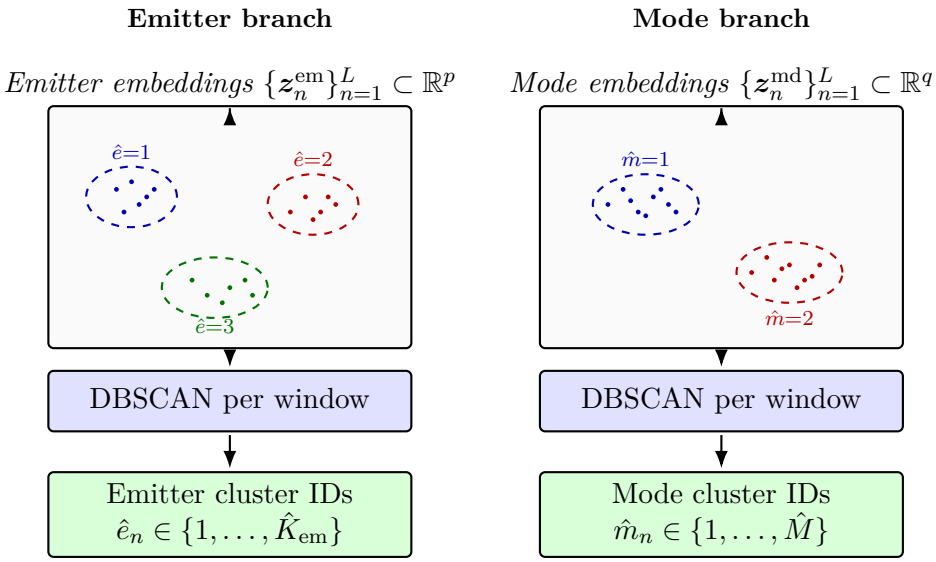


Figure 4.4. Inference-time decoding of branch embeddings into per-pulse cluster indices via Equation (4.14). Dots are pulse embeddings and dashed ellipses are DBSCAN clusters in one window. The pipeline is identical on both branches and the integer outputs are local to each clustering instance; they acquire their interpretation from the supervision scope of Section 4.1, which differs between the two branches.

4.4 Attention Variants

The encoder of Section 4.3 uses standard self-attention, which treats a window as an unordered set of tokens and lets every pulse attend to every other pulse purely through learned query–key compatibility. The only ordering cue available to it is whatever the input features themselves carry, in particular the TOA. A radar recording, however, has strong pairwise structure: two pulses that are close in time, share a carrier frequency, have a similar pulse width, or arrive from a similar direction, are far more likely to originate from the same emitter or mode than an arbitrary pair. This section describes two variants of the attention mechanism that make such structure explicit. They are used as drop-in modifications of the MHA sub-layers of Section 4.3. Each variant is evaluated separately against standard self-attention in Chapter 5; combining the bias with RoPE is not tested here.

Throughout, we write the attention logits of a single head over a window of L pulses as

$$\boldsymbol{A} = \frac{\boldsymbol{Q}\boldsymbol{K}^\top}{\sqrt{d_k}}, \qquad A_{ij} = \frac{\boldsymbol{q}_i^\top \boldsymbol{k}_j}{\sqrt{d_k}}, \tag{4.15}$$

where $\boldsymbol{q}_i$ and $\boldsymbol{k}_j$ are the query and key of pulses i and j and d_k is the per-head dimension. The attention weights are the row-wise softmax of $\boldsymbol{A}$, which then mixes the value vectors.

4.4.1 Physical attention bias

The first variant adds a pairwise bias to the attention logits, computed from the physical parameters of each pulse pair. The bias uses two coordinates: the window-local TOA t_n and the azimuth θ_n^{az} of Equation (4.7). For TOA the pairwise feature is the absolute difference $B_{ij}^{(t)} = |t_i - t_j|$. For azimuth, a plain absolute difference would treat bearings near 0 and 2π as far apart although they describe nearly the same direction; the circular distance

$$B_{ij}^{(\theta)} = \frac{1}{\pi} \min\Big(|\theta_i^{\mathrm{az}} - \theta_j^{\mathrm{az}}|,\, 2\pi - |\theta_i^{\mathrm{az}} - \theta_j^{\mathrm{az}}|\Big) \in [0, 1] \tag{4.16}$$

takes the shorter arc on the circle and normalises by π so that both bias features lie in $[0, 1]$. Each feature is added to the logits with a learnable scalar coefficient,

$$A_{ij} = \frac{\boldsymbol{q}_i^\top \boldsymbol{k}_j}{\sqrt{d_k}} + \lambda_t\, B_{ij}^{(t)} + \lambda_\theta\, B_{ij}^{(\theta)}. \tag{4.17}$$

Each coefficient is learned jointly with the rest of the network and belongs to a single head of a single attention sub-layer, so every head in every encoder block—throughout the shared trunk and both branches—carries its own $(\lambda_t, \lambda_\theta)$. This adds only 128 scalar parameters in total, given the layer and head counts of Section 4.3; the pairwise delta matrices themselves scale quadratically in the window length, as the attention mechanism already does. Coefficients may be negative, and each is initialised to a small negative value so that, in line with the intended inductive bias, a larger coordinate difference initially decreases attention between the corresponding pulses. Because the coefficients are learned independently in each part of the network, the branches can weight the cues differently. The deinterleaving branch is expected to rely more on azimuth, since pulses from one emitter usually arrive from a nearly constant bearing. The shared trunk and the mode branch should instead suppress that cue, because an operating mode must be recognised independently of viewing geometry. Whether this allocation emerges is examined in Chapter 5. The bias follows the same idea as additive pairwise attention biases in relative-position and graph Transformers [32, 33], but uses continuous pulse parameters instead of discrete token or path distances.

4.4.2 Rotary positional encoding

The bias of Equation (4.17) is added to the logits after the query–key product. The second variant uses the rotary encoding of Section 2.7, so the inner product depends on a pulse coordinate only through pairwise differences (Equation (2.24)).

In the original construction the coordinate is the token index. Consecutive pulses in a recording are not equally spaced, so an ordinal index misrepresents TOA gaps (Section 4.2). We attach physical coordinates instead, so that $p_j - p_i$ is a time gap or a difference of direction features.

Those coordinates lie in $[0, 1]$, whereas token indices advance in steps of one. A

scale $\gamma > 0$ therefore multiplies every frequency, so plane m is rotated by $\gamma p \omega_m$. Each coordinate has its own γ, reported in Chapter 5, and the base is $b = 10^4$ as in Su et al. [15]. The factor γ is left implicit in $\boldsymbol{R}$.

The encoder of Section 4.3 uses this construction in every attention sub-layer. The shared trunk and the mode branch rotate by the window-local TOA. The deinterleaving branch splits each head across two continuous functions of azimuth.

Trunk and mode branch

Every head in the shared trunk and in the mode branch rotates by the window-local TOA,

$$\tilde{\boldsymbol{q}}_n = \boldsymbol{R}(t_n)\, \boldsymbol{q}_n, \qquad \tilde{\boldsymbol{k}}_n = \boldsymbol{R}(t_n)\, \boldsymbol{k}_n, \tag{4.18}$$

with scale γ_t in $\boldsymbol{R}$. Arrival direction is excluded. An operating mode is characterized by the intrinsic temporal pattern of the emission and must be recognized regardless of viewing geometry. Rotating these layers by direction would make the mode embedding depend on that geometry.

Deinterleaving branch

Every head in the deinterleaving branch rotates by direction instead. Azimuth is a strong cue for emitter identity, and temporal structure is already present upstream through the TOA-rotated trunk. Feeding the raw angle θ_n^{az} into RoPE would reintroduce the 0–2π wrap of Section 4.2.1, because the relative phase of Equation (2.24) depends on the algebraic difference of the scalar coordinate. The branch therefore replaces θ_n^{az} by the two continuous features

$$\tilde{c}_n = \frac{1 + \cos\theta_n^{\mathrm{az}}}{2}, \qquad \tilde{s}_n = \frac{1 + \sin\theta_n^{\mathrm{az}}}{2}, \tag{4.19}$$

both in $[0, 1]$. The $d_k/2$ planes of each head are split evenly into two slices of $d_k/4$ planes, following multi-axis RoPE [34], so that the query (and likewise the key) decomposes as

$$\boldsymbol{q}_n = \begin{bmatrix} \boldsymbol{q}_n^{(c)} \\ \boldsymbol{q}_n^{(s)} \end{bmatrix}, \qquad \boldsymbol{q}_n^{(c)},\, \boldsymbol{q}_n^{(s)} \in \mathbb{R}^{d_k/2}. \tag{4.20}$$

Each half is rotated by its own single-coordinate matrix of the form Equation (2.22),

$$\boldsymbol{R}_n^{\mathrm{em}} = \mathrm{diag}\Big(\boldsymbol{R}^{(c)}(\tilde{c}_n),\ \boldsymbol{R}^{(s)}(\tilde{s}_n)\Big), \tag{4.21}$$

with $\boldsymbol{R}^{(c)}, \boldsymbol{R}^{(s)} \in \mathbb{R}^{d_k/2 \times d_k/2}$ using scales γ_c and γ_s respectively. The rotated query is

$$\tilde{\boldsymbol{q}}_n = \boldsymbol{R}_n^{\mathrm{em}} \boldsymbol{q}_n = \begin{bmatrix} \boldsymbol{R}^{(c)}(\tilde{c}_n)\, \boldsymbol{q}_n^{(c)} \\ \boldsymbol{R}^{(s)}(\tilde{s}_n)\, \boldsymbol{q}_n^{(s)} \end{bmatrix}, \tag{4.22}$$

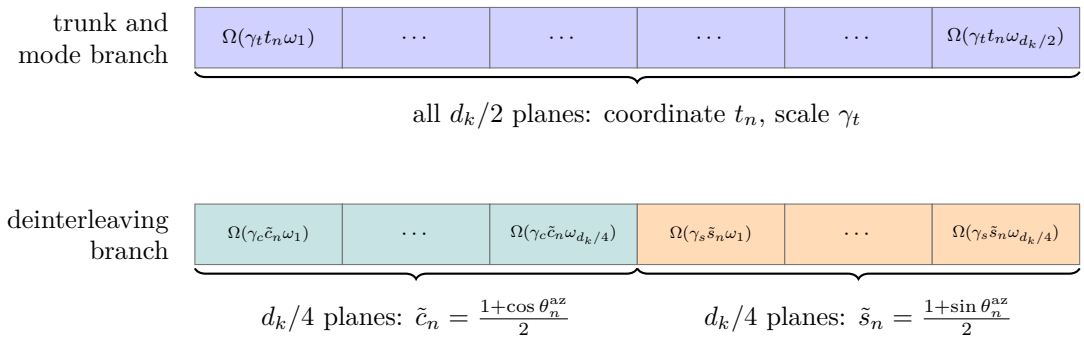


Figure 4.5. Allocation of the $d_k/2$ rotation planes of one attention head. Top: shared trunk and mode branch, all planes rotated by the TOA t_n with scale γ_t (Equation (4.18)). Bottom: deinterleaving branch, planes split evenly between $\tilde{c}_n$ and $\tilde{s}_n$ of Equation (4.19), with scales γ_c and γ_s (Equations (4.21) and (4.23)). Ellipsis cells are intermediate planes of each frequency ladder.

and likewise for the key. Applying Equation (2.24) slice-wise yields

$$\left\langle \tilde{\boldsymbol{q}}_i,\, \tilde{\boldsymbol{k}}_j \right\rangle = \boldsymbol{q}_i^{(c)\top} \boldsymbol{R}^{(c)}\left(\tilde{c}_j - \tilde{c}_i\right) \boldsymbol{k}_j^{(c)} + \boldsymbol{q}_i^{(s)\top} \boldsymbol{R}^{(s)}\left(\tilde{s}_j - \tilde{s}_i\right) \boldsymbol{k}_j^{(s)}. \tag{4.23}$$

Two bearings just above 0 and just below 2π are almost the same direction, but their raw angles differ by nearly a full turn. Their $(\tilde{c}, \tilde{s})$ values are almost the same, so the relative phase in Equation (4.23) stays small. An algebraic gap in θ^{az} itself would not.

A control that rotates every layer of the trunk and of both branches by window-local TOA, with no azimuth encoding, is evaluated in Chapter 5. That uniform layout tests whether the branch-specific allocation above is necessary, or whether a single temporal coordinate already suffices.

Figure 4.5 summarizes the plane allocation for the trunk/mode and deinterleaving layouts.

4.5 Loss Function

The training objective is a sum of two supervised contrastive [24] terms with the same functional form, both aggregated over the whole mini-batch. The two terms differ only in which label drives the positive set: an emitter identifier on the emitter branch, and the global mode label of Section 4.1 on the mode branch. Both branches use a contrastive loss rather than a closed-catalogue softmax, so training pushes pulses with the same label together in embedding space instead of scoring a fixed set of class logits. That encourages the encoder to learn general similarity structure among pulses. Inference then clusters those embeddings (Equation (4.14)); the clustering step is not part of the loss and contributes no gradients. In principle the same embeddings could also support open-set modes or a later classification head on a library of known emitters using the outputted encodings, but those uses are not evaluated in this thesis.

Supervised contrastive kernel

The projection-head outputs of Equation (4.12) are unit-norm by construction, so that the cosine similarity reduces to the inner product

$$s_{ij} \;\coloneqq\; \boldsymbol{z}_i^{\top}\boldsymbol{z}_j \;\in\; [-1,1], \tag{4.24}$$

and a fixed temperature $\tau > 0$ rescales it inside the loss. Smaller τ concentrates the gradient on the hardest negatives, while larger τ stabilizes early optimization, since $\|\nabla\mathcal{L}_{\mathrm{SC}}\| \propto \frac{1}{\tau}$ [24]. Both branches use $\tau = 0.5$, the value suggested in the original supervised contrastive formulation [24].

Let a mini-batch consist of B windows of length L, and write $\mathcal{B}$ for the set of all $B \times L$ pulse embeddings pooled across those windows. How the windows are chosen is controlled by the scenario-aware sampler of Section 4.6. For each pulse $i \in \mathcal{B}$, write the candidate set $A(i) = \mathcal{B} \setminus \{i\}$ and the positive subset $P(i) \subseteq A(i)$ defined by label coincidence. The supervised contrastive contribution [24] is then

$$\mathcal{L}_{\mathrm{SC}} \;=\; -\sum_{i\in\mathcal{B}^{+}} \frac{1}{|P(i)|} \sum_{p\in P(i)} \log \frac{\exp(s_{ip}/\tau)}{\sum_{a\in A(i)} \exp(s_{ia}/\tau)}, \tag{4.25}$$

where $\mathcal{B}^{+} = \{i \in \mathcal{B} : P(i) \neq \emptyset\}$ and $\mathcal{L}_{\mathrm{SC}} = 0$ when $\mathcal{B}^{+} = \emptyset$. Pulses outside $\mathcal{B}^{+}$ contribute only as negatives. The denominator runs over every other pulse in the batch, so unrelated samples exert repulsive pressure and the encoder cannot collapse to a constant map [11].

Branch losses and total objective

Both branches apply Equation (4.25) to the same batch embeddings and differ only in the label that defines the positive sets: two pulses are positives on the emitter branch when they originate from the same physical emitter, and on the mode branch when they carry the same global mode label. Since emitter identifiers are recording-local in Section 4.1, they are made unique within a mini-batch by pairing each identifier with its originating recording. Writing $\mathcal{L}_{\mathrm{em}}$ and $\mathcal{L}_{\mathrm{md}}$ for the resulting losses on the emitter and mode projections, the total training objective is

$$\mathcal{L} \;=\; \mathcal{L}_{\mathrm{em}} + \lambda_{\mathrm{md}}\,\mathcal{L}_{\mathrm{md}}, \qquad \lambda_{\mathrm{md}} = 1, \tag{4.26}$$

so the two branches carry equal weight. Tuning λ_{md} could improve the trade-off between the tasks, but is not evaluated in this thesis. The factor $1/|P(i)|$ in Equation (4.25) keeps each pulse's contribution intensive in its positive-set size, which matters because emitter positives are usually few while mode positives can be many; a Gibbs–Boltzmann reading of the same kernel is given in Chapter D.

4.6 Training Procedure

Parameters $\boldsymbol{\theta}$ are estimated by minimising $\mathcal{L}$ (Equation (4.26)) with AdamW [37] in PyTorch. Training runs for eight epochs on the training split; after each epoch the model is evaluated on the validation split. The learning rate rises linearly from zero to its peak over the first 5% of the total optimisation steps (eight epochs times the number of training batches), then follows a cosine anneal. Peak learning rate, weight decay, batch size, and related settings are listed in Table 5.2.

Scenario-aware batch sampling

Mini-batches are not formed by uniform shuffling of the pooled windows. Emitter identifiers are recording-local (Section 4.1), so two windows drawn from different scenarios never share an emitter positive even when both contain pulses from physically similar sources. Uniform mixing across the corpus would therefore leave the emitter branch with positives only inside each individual window, unless two windows happen to be drawn from the same scenario—an event that is unlikely among roughly 1500 training scenarios (Section 5.2). Such sparse positives can hinder learning and leave most of a multi-window batch unused for the emitter term.

Each mini-batch is instead drawn from a single scenario, with several windows taken from that scenario so that the same emitter can appear across windows and supply additional positives. Mode labels remain globally comparable across scenarios, but drawing several windows from one scenario also increases the number of same-mode positives available to the mode branch in each batch. The number of windows per batch is recorded in Table 5.2.

4.7 Evaluation Metrics

Inference returns, for each window, an emitter labelling $(\hat{e}_1, \ldots, \hat{e}_L)$ and a mode labelling $(\hat{m}_1, \ldots, \hat{m}_L)$ (Equation (4.14)). These are compared with the simulator labels (e_n, m_n) of Equation (4.1) on the same window. Cluster names are assigned independently per window and per branch, so only scores that are invariant to relabelling are valid. The definitions are those of Section 2.5.

Each branch is scored separately: emitter agreement from $(e_n, \hat{e}_n)$ and mode agreement from $(m_n, \hat{m}_n)$. Global mode labels enter training as an inductive bias that pulls same-mode embeddings together. They are not a catalogue of names at inference, and the mode score remains a partition comparison.

The reported scores are ARI, AMI, homogeneity, and completeness, computed on every test window. ARI summarises pair agreement and stays comparable when the number of active emitters or modes changes between windows. AMI measures how much one labelling tells about the other. It is less dominated by the largest clusters. Pulse

counts per emitter are often unbalanced, so that difference is useful here. Homogeneity and completeness are reported separately rather than only through the V-measure. A high homogeneity with low completeness means true emitters or modes were split; the reverse means distinct sources were merged.

Two-dimensional projections of $\{\boldsymbol{z}_n^{\mathrm{em}}\}$ and $\{\boldsymbol{z}_n^{\mathrm{md}}\}$, coloured by (e_n, m_n), show whether same-label pulses sit together in each embedding. Protocols and numbers are in Chapter 5.

4.8 Baseline Methods

The comparisons isolate the contribution of the mode term $\lambda_{\mathrm{md}}\mathcal{L}_{\mathrm{md}}$ in Equation (4.26) from that of the attention variants in Section 4.4, relative to standard self-attention. Final configurations and numbers are deferred to Chapter 5; this section fixes only the design intent of each control.

Dual-branch, single-task. The encoder of Section 4.3 with standard self-attention, trained with one contrastive term only: $\mathcal{L}_{\mathrm{em}}$ (emitter-only) or $\mathcal{L}_{\mathrm{md}}$ (mode-only). Architecture, width, and decoding remain those of the joint model, so the only change is the objective. Emitter-only is the comparable transformer for whether the joint loss outperforms deinterleaving-alone training; mode-only completes the symmetric picture. Both branches are still decoded with DBSCAN, including the branch that received no direct gradient.

DBSCAN on fixed window features. No transformer; DBSCAN [5] clusters a deterministic vector built from the scaled PDWs of a window. The neighbourhood radius ε is selected by grid search on the validation set, holding minPts fixed; the selected ε is then frozen for the held-out test evaluation. Separates the gain from learned sequence context from what density clustering already recovers on hand-specified summaries. This control is not included in the tables of Chapter 5 unless a corresponding run is added.

CHAPTER 5

Experiments and Results

5.1 Experimental Setup

All neural models in this chapter were trained and evaluated on a single workstation with one NVIDIA A100 GPU (80 GB high-bandwidth memory). The implementation is written in Python with PyTorch. The source code and training scripts remain proprietary to Saab and are not released with this thesis.

Fixed random seeds control weight initialization, training-window shuffling, and other stochastic steps, so that ablations in Section 5.5 are run under identical noise conditions unless an experiment deliberately varies the seed.

5.2 Datasets

All experiments in this chapter use synthetic PDW streams generated by the proprietary scenario simulator introduced in Section 4.2. The simulator supplies time-ordered pulse records together with ground-truth labels for emitter identity (meaningful only within one scenario, in the sense of Section 4.1) and operating mode (drawn from the global catalogue); both label streams are consumed by the training objective in Section 4.5 and are also used to compute the evaluation metrics in Sections 4.7 and 5.4.

Each *scenario* is one simulated timeline: a sequence of PDWs whose length and the number of concurrent emitters vary across draws so that the encoder sees both sparse and crowded electromagnetic pictures. Scenarios are disjointly assigned to training, validation, and test partitions before any windowing; training-split statistics for carrier frequency, pulse width, and amplitude in Section 4.2 are fit on the training partition only and applied unchanged to validation and test pulses, whereas TOA is min–max mapped independently inside each window. Windows contain $L = 2000$ pulses; training windows use stride $\delta = 200$, so consecutive windows overlap by $L - \delta = 1800$ pulses, whereas validation and test windows use stride $\delta = L$ and are therefore non-overlapping. As a consequence, the effective number of windows per scenario differs by split even when raw pulse counts are similar.

Table 5.1 summarises corpus-level figures. The training split contains 1461 scenarios and roughly 13.8 million pulses in total, while the validation and test splits contribute approximately 2.25 and 2.36 million pulses over 245 and 244 scenarios, respectively. The emitter column counts the distinct emitters active in each scenario. Emitter identifiers are meaningful only within their own scenario (Section 4.1), so these figures describe how many emitters each scenario contains rather than membership in some global

Table 5.1. Synthetic scenario corpus used for training and evaluation. Per-scenario quantities are reported as mean ± standard deviation over the scenarios in each split.

		Per scenario (mean ± std)		
Split	Scenarios	Pulses	Emitters	Modes
Train	1461	9480 ± 1690	8.80 ± 4.33	4.33 ± 1.40
Validation	245	9180 ± 2240	8.59 ± 4.41	4.28 ± 1.57
Test	244	9670 ± 1270	8.93 ± 4.56	4.39 ± 1.49

emitter set; at inference, the within-window clustering accordingly never has to separate more emitters than the enclosing scenario contains. The number of unique emitters per scenario ranges from 1 to 23 and the number of unique modes per scenario from 1 to 7 in every split, so the three partitions are statistically homogeneous in this respect. The global mode catalogue comprises $M = 7$ operating modes (Section 4.1), all of which are exercised in each split.

The numbers of emitters and assigned modes per scenario are sampled uniformly; however, the resulting pulse counts per emitter and mode are highly imbalanced due to variation in mode behavior. For instance, scanning modes typically generate far fewer pulses than lock-on modes, and some modes differ by more than an order of magnitude in their base PRI. As a result, the number of pulses (PDWs) per emitter or mode is not balanced either within a scenario or across the corpus. Because the encoder sees only windows of fixed length L (Section 4.2), rare combinations of emitter, mode, and interleaving pattern can be missing from individual windows or represented by only a handful of pulses, which limits what any single forward pass can evidence about those labels.

5.2.1 Qualitative properties of the synthetic corpus

Beyond aggregate counts, the corpus is checked qualitatively before large training runs: example timelines are inspected for plausible PRI structure per mode, for physically admissible RF and pulse-width ranges, and for the intended mixture density when multiple emitters are active. When the simulator regime adds dropped or spurious pulses (Section 4.2), spot checks confirm that surviving pulses retain their original time order and that spurious insertions stay within the configured parameter ranges. This mirrors the reporting practice in earlier Saab-hosted simulator studies, which often include illustrative emitter tracks alongside tables [20]. Section 5.6 complements the brief sanity checks here with embedding-space visualizations and other qualitative diagnostics once model checkpoints are available.

Table 5.2. Primary optimisation hyperparameters for the proposed model.

Setting	Value
Optimiser	AdamW [37] with PyTorch defaults $(\beta_1, \beta_2) = (0.9, 0.999)$ and $\epsilon = 10^{-8}$
Peak learning rate $\eta_{\max}$	—
Warmup	Linear from 0 to $\eta_{\max}$ over 5% of total steps
Schedule after warmup	Cosine annealing to $\eta_{\min}$
Minimum learning rate $\eta_{\min}$	0
Epochs	8
Dropout (attention and feed-forward)	0.1 throughout the transformer blocks
Batch size B (windows per step)	—
Mode loss weight λ_{md}	1
Weight decay	—

Table 5.3. Rotary scales and DBSCAN radii. ε_{em} and ε_{md} are selected independently per model on the validation set; minPts = 5 is fixed.

Model	γ_t	γ_c	γ_s	ε_{em}	ε_{md}
Vanilla	n/a	n/a	n/a	—	—
RoPE-TOA	—	n/a	n/a	—	—
RoPE-az	—	—	—	—	—
Bias	n/a	n/a	n/a	—	—

5.3 Hyperparameters and Training Details

This section collects numerical training choices deferred from Sections 4.3, 4.5 and 4.6. Unless Section 5.5 states otherwise, baseline models use the same epoch budget and hardware (Section 5.1) so that differences reflect architecture and objective rather than compute.

The contrastive temperature is fixed at $\tau = 0.5$ as in Section 4.5. Table 5.2 summarises the optimisation schedule and regularisation for the proposed dual-branch encoder; entries marked with an em dash will be filled before the final thesis submission.

Total optimisation steps equal eight epochs times the number of training batches per epoch. The warmup fraction and cosine phase are counted in those steps, as stated in Section 4.6.

Table 5.3 collects the attention-variant knobs deferred from Section 4.4 and the DBSCAN radii of Section 4.3. Vanilla and Bias do not use rotary scales. RoPE-TOA uses a single scale γ_t on every layer; RoPE-az uses γ_t on the trunk and the mode branch and (γ_c, γ_s) on the deinterleaving branch. Unused coordinates are marked n/a; an em dash in an ε column, or in a γ column that the model does use, will be replaced after selection.

The single-task runs of Section 5.5 inherit the Vanilla row of Tables 5.2 and 5.3, including a re-selected ε per branch on the validation set of each objective.

5.4 Quantitative Results

This section reports clustering scores for four jointly trained encoders that differ only in the attention mechanism. The Vanilla encoder, with standard self-attention, is the existence test for RQ1; the remaining three rows are the RQ3 comparison against that baseline. The joint-versus-single-task comparison that answers RQ2 is deferred to Section 5.5. Numerical entries marked with an em dash, and the figures under `figures/experiments/`, will be filled after the test evaluation.

5.4.1 Evaluation protocol

All scores are computed on the held-out test split of Section 5.2. Windows have length $L = 2000$ and stride $\delta = L$, so they do not overlap. For each window, DBSCAN on each branch yields a predicted partition that is scored against the simulator labels with ARI, AMI, homogeneity, and completeness (Section 4.7).

Each table entry is the mean $\pm$ one standard deviation of that per-window score over the test windows. Results are from a single training seed; the reported spread is therefore window-level variation, not seed-level uncertainty. The DBSCAN radius ε is selected independently for each model and each branch on the validation grid of Section 4.3, with minPts $= 5$ throughout; selected radii are listed in Table 5.3.

Homogeneity and completeness are reported separately rather than only through their harmonic mean, the V-measure, so that oversplitting can be distinguished from undermerging. Mean absolute cluster-count error $|\hat{K} - K|$ (emitters) and $|\hat{M} - M|$ (modes) summarises how well the inferred number of clusters matches the number of distinct reference labels in the same window.

5.4.2 Attention variants

Four configurations are trained with the joint objective of Equation (4.26) and otherwise identical hyperparameters (Section 5.3). Vanilla is the encoder of Section 4.3 with standard MHA; TOA enters only as an input feature. RoPE-TOA applies rotary encoding on the window-local TOA in the trunk and in both branches. RoPE-az is the branch-specific design of Section 4.4: TOA on the trunk and the mode branch, and the wrap-safe pair $(\tilde{c}, \tilde{s})$ of Equation (4.19) on the deinterleaving branch. Bias adds the pairwise physical logits of Equation (4.17). No model combines RoPE with the bias.

Vanilla tests whether a transformer encoder can deinterleave pulses and recover operating modes by clustering (RQ1). The remaining three rows test whether explicit physical structure in attention improves those scores relative to standard MHA (RQ3).

Table 5.4. Emitter clustering on the test split. Entries are mean ± std over non-overlapping test windows from a single training seed. Em dashes will be replaced after evaluation; the best value in each column will then be set in bold.

Model	ARI	AMI	Homogeneity	Completeness	$\lvert\hat{K} - K\rvert$
Vanilla	—	—	—	—	—
RoPE-TOA	—	—	—	—	—
RoPE-az	—	—	—	—	—
Bias	—	—	—	—	—

Table 5.5. Mode clustering on the test split. Entries follow the same aggregation as Table 5.4.

Model	ARI	AMI	Homogeneity	Completeness	$\lvert\hat{M} - M\rvert$
Vanilla	—	—	—	—	—
RoPE-TOA	—	—	—	—	—
RoPE-az	—	—	—	—	—
Bias	—	—	—	—	—

Tables 5.4 and 5.5 collect the window-averaged scores; the best value in each column will be set in bold once the numbers are filled.

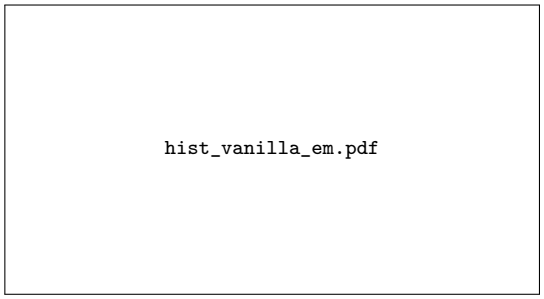
The histograms in Figures 5.1 and 5.2 show the distribution of the four metrics over test windows, one panel per model. Each panel is the four-plot already produced per run (ARI, AMI, homogeneity, completeness). A mass near one indicates consistent recovery. A left tail indicates windows where clustering fails, which in this corpus typically coincide with crowded mixtures or rare-label windows (Section 5.2).

5.4.3 Cluster-count recovery

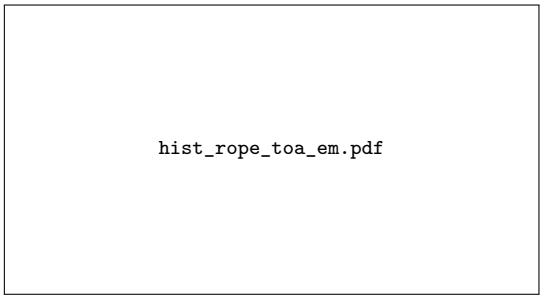
Figures 5.3 and 5.4 scatter, for each test window, the number of clusters returned by DBSCAN against the number of distinct reference labels in that window. The identity line $y = x$ is exact count recovery. Points above the line oversplit a true class, which lowers completeness; points below it merge distinct classes, which lowers homogeneity. That geometry is why Tables 5.4 and 5.5 report homogeneity and completeness separately, and why the mean absolute count error is included as a summary column.

5.4.4 Model size and inference cost

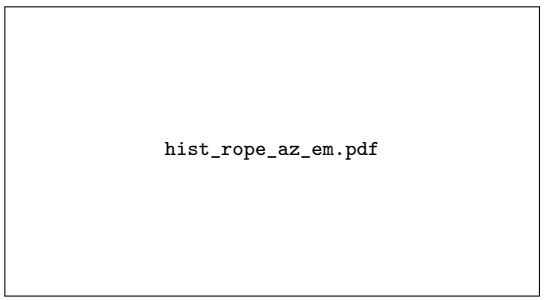
Table 5.6 reports the number of trainable parameters and the wall-clock time per test window. Timing is measured on the A100 of Section 5.1, with one window per forward pass, and includes both the encoder and the DBSCAN decoding of Equation (4.14). A short GPU warmup precedes the timed loop and is excluded from the mean. RoPE adds no trainable weights relative to Vanilla; Bias adds the 128 scalar coefficients of Section 4.4.



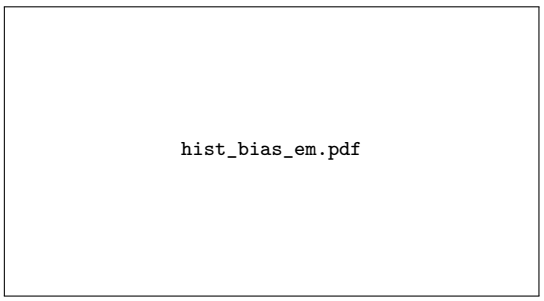
(a) Vanilla



(b) RoPE-TOA

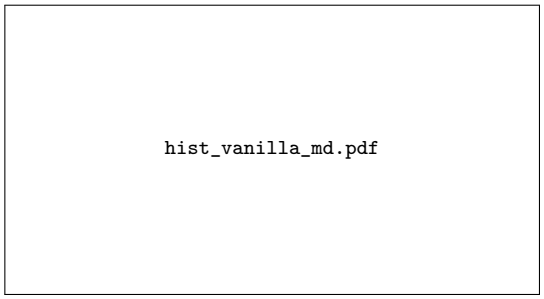


(c) RoPE-az

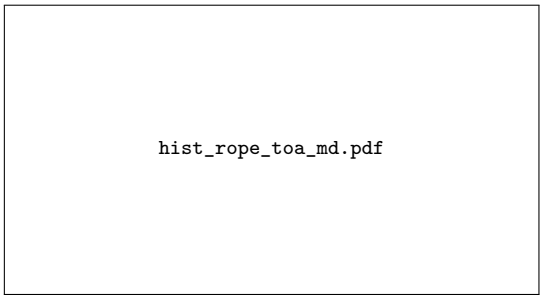


(d) Bias

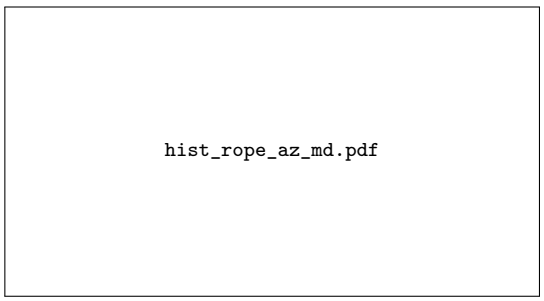
Figure 5.1. Distribution of emitter-clustering metrics over non-overlapping test windows. Each panel is a four-plot of ARI, AMI, homogeneity, and completeness for one attention variant.



(a) Vanilla



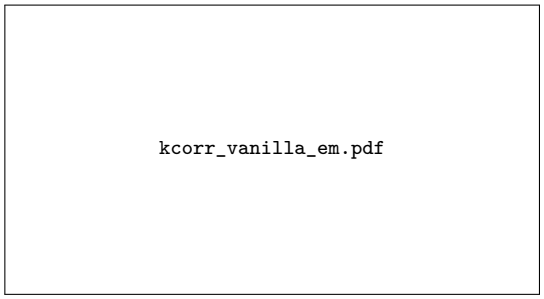
(b) RoPE-TOA



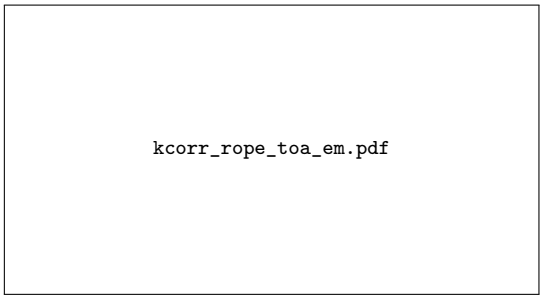
(c) RoPE-az

(d) Bias

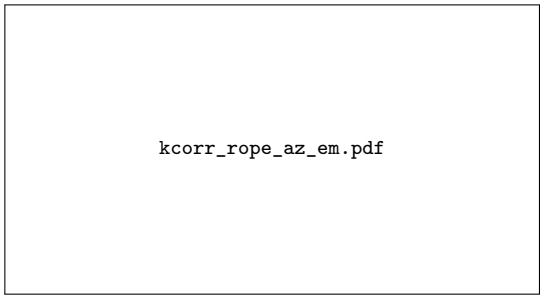
Figure 5.2. Distribution of mode-clustering metrics over non-overlapping test windows. Layout as in Figure 5.1.



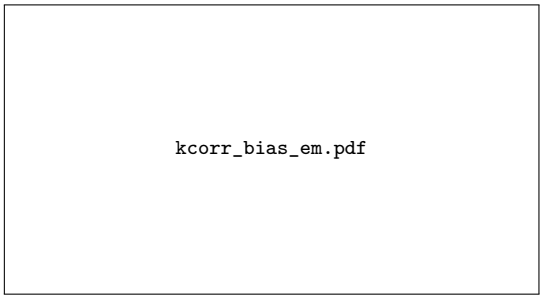
(a) Vanilla



(b) RoPE-TOA

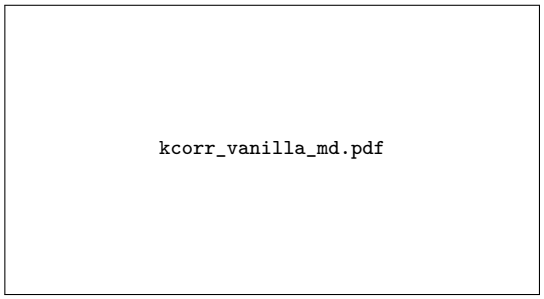


(c) RoPE-az

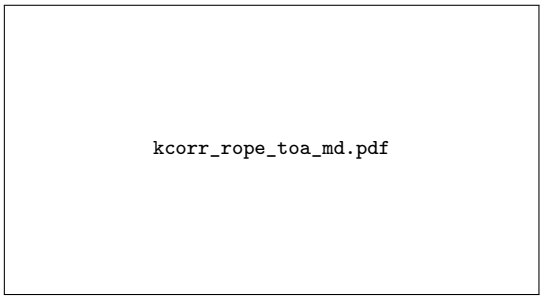


(d) Bias

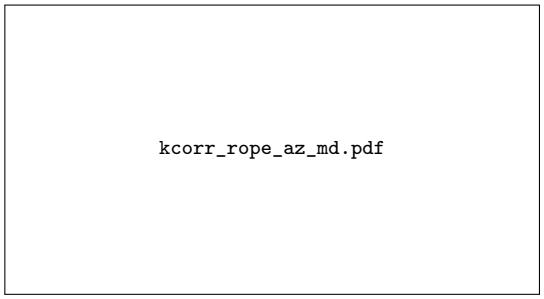
Figure 5.3. Predicted versus true number of emitters per test window. The line $y = x$ is exact count recovery; points above it oversplit, points below it undermerge.



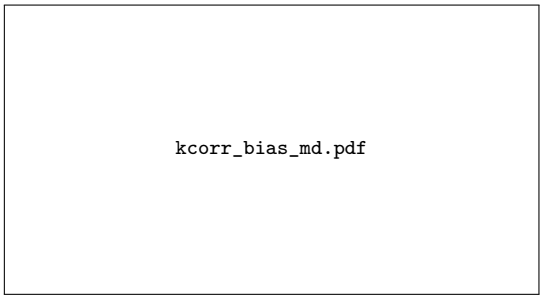
(a) Vanilla



(b) RoPE-TOA



(c) RoPE-az



(d) Bias

Figure 5.4. Predicted versus true number of modes per test window. Layout as in Figure 5.3.

Table 5.6. Trainable parameters and inference time per test window (mean ± std), including DBSCAN decoding. Em dashes will be replaced after evaluation.

Model	Parameters	Time (ms/window)
Vanilla	—	—
RoPE-TOA	—	—
RoPE-az	—	—
Bias	—	—

5.5 Ablation Studies

This section isolates the joint objective of Equation (4.26) from the attention variants of Section 5.4.2. All three runs use the Vanilla encoder of Section 4.3, so the only change is which contrastive term is active. That matched dual-branch control is the comparable transformer for RQ2: whether coupling deinterleaving with mode clustering outperforms training for one task alone.

The three objectives are:

Joint $\mathcal{L} = \mathcal{L}_{\text{em}} + \lambda_{\text{md}}\mathcal{L}_{\text{md}}$ with $\lambda_{\text{md}} = 1$, identical to Vanilla in Section 5.4.

Emitter-only $\mathcal{L} = \mathcal{L}_{\text{em}}$; the mode branch receives no direct gradient.

Mode-only $\mathcal{L} = \mathcal{L}_{\text{md}}$; the emitter branch receives no direct gradient.

Joint numbers in Table 5.7 are copied from the Vanilla row of Tables 5.4 and 5.5, not from a second training run. The single-task models are trained from the same initialisation seed, on the same data splits, and with the remaining hyperparameters of Section 5.3.

Both branches are decoded with DBSCAN in every run, including the branch that was not trained. The informative cells are therefore the emitter scores of Mode-only and the mode scores of Emitter-only: they measure how much the shared trunk transfers to the unsupervised branch, versus collapse of that branch. RQ2 is answered primarily by comparing Joint with Emitter-only on the emitter metrics; the mode column and the Mode-only row complete the symmetric picture.

Count error is $|\hat{K} - K|$ on the emitter branch and $|\hat{M} - M|$ on the mode branch. No additional histogram or scatter pages are included here; the distributions for Joint are those of Vanilla in Figures 5.1 to 5.4.

5.6 Qualitative Analysis

Qualitative analysis complements the scalar metrics in Section 5.4 by checking whether embeddings organize pulses in ways that align with physical emitter boundaries and with mode transitions inside a window. The checks in Section 5.2.1 validate that the

Table 5.7. Joint versus single-task objectives on the Vanilla encoder. Aggregation as in Tables 5.4 and 5.5. Hom. and Comp. are homogeneity and completeness. Joint entries are those of Vanilla, not a repeated run. Em dashes will be replaced after evaluation.

Objective	Branch	ARI	AMI	Hom.	Comp.	Count error
Joint	Emitter	—	—	—	—	—
Joint	Mode	—	—	—	—	—
Emitter-only	Emitter	—	—	—	—	—
Emitter-only	Mode	—	—	—	—	—
Mode-only	Emitter	—	—	—	—	—
Mode-only	Mode	—	—	—	—	—

simulator export behaves as intended; this section applies analogous scrutiny *after* models are trained.

Two-dimensional projections of pooled or per-pulse embeddings (for example UMAP or t-SNE) should be colored separately by reference emitter (using the recording-local emitter label of each pulse) and by reference mode (using the global mode catalogue from Section 4.1). Disjoint emitter-colored manifolds with mode-colored substructure that stays coherent within an emitter are evidence that the dual geometry in Chapter 4 is behaving as designed; elongated bridges between emitter manifolds often indicate residual deinterleaving ambiguity or shared waveform parameters across platforms.

Attention maps or head-wise statistics, if exported, can be overlaid on TOA-ordered pulses to see whether specific heads track stable intervals versus rapid mode hops.

Confusion matrices between inferred clusters and reference labels (after optimal assignment) are optional but useful when a small number of emitter or mode classes dominate error mass.

CHAPTER 6

Discussion

6.1 Interpretation of Results

This section connects the quantitative tables in Section 5.4 and the qualitative material in Section 5.6 to the modeling choices in Chapter 4. The goal is not to restate individual scores, but to explain *which* mechanisms plausibly drive the observed ranking of methods and ablations.

When emitter-level AMI or ARI lag while mode-level scores are stronger, a natural hypothesis is that the encoder prioritizes intra-window structure tied to waveform parameters (which modes share) before it fully exploits slower cues that separate physical platforms across mixed traffic. Conversely, if emitter scores dominate but mode partitions collapse, the dual-head coupling or the mode contrastive term may be underweighted relative to the emitter branch, or certain modes may be too short-lived inside length-L windows to support stable clusters.

The dropped and spurious-pulse regimes produced by the simulator (Section 4.2) interact with self-attention in two ways: they perturb local timing patterns that define modes, and they change which pulses co-occur inside a window, which can confuse any window-level emitter summary. Interpretation should therefore read noise sweeps alongside the corpus diagnostics in Sections 5.2.1 and 5.6: a metric drop that coincides with visibly corrupted tracks is evidence of a robustness limit, whereas a drop on clean held-out scenarios points toward generalization or optimization issues instead.

6.2 Comparison with Related Work

Relative to classical ELINT fingerprinter pipelines surveyed in Section 3.1, the proposed approach trades hand-engineered feature logic for a learned encoder, at the cost of data hunger, compute, and reduced transparency unless accompanied by diagnostics such as those in Section 5.6. Unlike the supervised radar classifiers discussed in Section 3.2, which typically assume a fixed global label set for emitters and a fixed mode catalogue at inference, the training objective here treats emitter identity as a *recording-local* symbol used only through within-recording comparisons (Section 4.5); operating-mode labels are used globally during training to shape the mode embedding geometry, but the inference path runs through clustering rather than through a softmax over the training catalogue, so that the same model can accommodate modulation types absent from the training data.

Deep clustering methods in Section 3.3 typically optimise a *single* flat partition

without labels, or attach two unrelated supervised heads to the same backbone. This thesis instead targets two aligned partitions with a nesting interpretation (modes within emitters), with two supervised contrastive losses that share the same functional form and differ only in the scope of their positive sets: within-window for emitter labels and batch-wide for global mode labels. The setup is closer in spirit to offline pulse-level grouping work that reasons about geometry and overlap in ESM data [18] but uses gradient-based representation learning rather than hand-tuned density stages alone. Where scores fall short of the best published supervised benchmarks, the gap is expected: those models often operate on curated single-emitter tracks with a flat, globally consistent emitter label set, whereas the present setting stresses mixture windows and a label structure that forces emitter inference through clustering rather than through a global classifier.

6.3 Limitations

The experimental programme is offline: models are trained and scored on bounded, revisit-able windows rather than under the latency, memory, and non-stationarity constraints of a continuously arriving pulse stream (Sections 1.4, 1.4.2 and 4.2). That setting supports controlled optimization and reproducible metrics but does not demonstrate that the same representations would support stable online emitter or mode tracking without periodic retraining or revised windowing policy.

Fixed-length windows are partly a consequence of full self-attention, whose standard implementation scales quadratically with the number of pulses in the window (Section 4.2). The chosen cap on L therefore trades faithful coverage of long mode segments and rare emitters against compute, and the present pipeline drops trailing segments shorter than L, which can remove evidence entirely when a scenario does not fill an integral number of windows.

Interleaved emitters and mode-hopping further skew how many pulses from each ground-truth label appear inside any given window, so performance summaries aggregate over heterogeneous label incidence rather than over an idealized balanced design (Section 5.2). Where metrics depend on comparing inferred partitions to reference labels, conclusions are most informative for label configurations that actually recur with sufficient mass inside windows; weak support for a label in a window is a limitation of the observation model, not necessarily of the encoder in isolation.

Finally, all quantitative evidence is derived from synthetic scenarios with full reference labels; operational recordings with imperfect deinterleaving, sensor dropouts, and distribution shift are not included here. Related Saab-line work on track association under deinterleaving errors highlights that real pipelines fragment pulse trains and introduce structured mistakes that are only partly captured by the stylized drop and spurious models used for stress tests [38]. Likewise, classifiers trained on simulator draws can exhibit overconfidence on held-out emitters or modulation families, which motivates explicit out-of-distribution and drift analyses when moving toward fielded

receivers [21]. The present thesis does not implement drift monitors or uncertainty heads; those remain orthogonal safeguards if embeddings from Chapter 4 are ever deployed under evolving threat libraries.

The DBSCAN post-processing used in Section 4.3 also encodes density assumptions that may not match every deployment, and we do not exhaust efficient-attention or hierarchical alternatives that could extend context within the same hardware envelope.

6.4 Implications

For ELINT and ESM practice, the main takeaway is methodological: joint emitter- and mode-aware embeddings can be learned from PDW streams whose supervision is structured asymmetrically (recording-local emitter labels, global mode catalogue), with downstream discretisation handled by clustering on *both* branches so that previously unseen emitters and modulation types can in principle surface as distinct clusters rather than be silently absorbed into existing catalogue entries. Operators should treat the resulting per-pulse emitter and mode cluster indices as hypotheses that require human or library-based adjudication before tactical use, especially when training data are synthetic and drift relative to live emitters or to mode catalogues is unmodelled; in particular, mode-branch clusters whose count exceeds the size of the training catalogue, or that occupy regions of the mode embedding space far from those populated by training-time modes, are informative as candidate novel modulations and warrant separate review.

For the deep-clustering and sequence-modeling research communities, the thesis sits at the intersection of contrastive representation learning on irregular pulse sequences and multi-partition clustering with a physical nesting constraint (modes within emitters). The architectural and loss choices in Chapter 4 are portable to other domains with hierarchical latent structure, provided that evaluation metrics separate which level of the hierarchy is being scored.

Follow-up work naturally includes streaming or sliding-window evaluation, richer deinterleaving and track-error models [38], and explicit ML safeguards against dataset shift and overconfident extrapolation [21]. On the ethics side, the considerations in Section 1.6 still apply: dual-use sensitivity, data stewardship if operational traces are introduced later, and the energy cost of transformer-scale training should be revisited whenever the experimental footprint grows.

CHAPTER 7

Conclusion

7.1 Summary

Placeholder paragraph.

7.2 Answers to the Research Questions

Placeholder paragraph.

7.3 Future Work

Placeholder paragraph.

References

[1] M. I. Skolnik, *Introduction to Radar Systems*, 3rd ed. McGraw-Hill, 2001.

[2] R. G. Wiley, *ELINT: The Interception and Analysis of Radar Signals*. Artech House, 2006.

[3] M. I. Skolnik, *Radar Handbook*, 3rd ed. McGraw-Hill, 2008.

[4] Z. Qu, J. Zhang, Y. Zhou, and L. Ni, "The intelligent evolution of radar signal deinterleaving: A systematic review from foundational algorithms to cognitive AI frontiers", *Sensors*, vol. 26, no. 1, 2026. DOI: 10.3390/s26010248

[5] M. Ester, H.-P. Kriegel, J. Sander, and X. Xu, "A density-based algorithm for discovering clusters in large spatial databases with noise", in *Proceedings of the 2nd International Conference on Knowledge Discovery and Data Mining (KDD)*, 1996, pp. 226–231.

[6] E. Gunn, A. Hosford, D. Mannion, J. Williams, V. Chhabra, and V. Nockles, *Radar pulse deinterleaving with transformer based deep metric learning*, arXiv preprint arXiv:2503.13476, 2025. arXiv: 2503.13476 [cs.LG].

[7] J. Xie, R. Girshick, and A. Farhadi, "Unsupervised deep embedding for clustering analysis", in *International Conference on Machine Learning (ICML)*, 2016, pp. 478–487.

[8] L. Hubert and P. Arabie, "Comparing partitions", *Journal of Classification*, vol. 2, no. 1, pp. 193–218, 1985.

[9] N. X. Vinh, J. Epps, and J. Bailey, "Information theoretic measures for clusterings comparison: Variants, properties, normalization and correction for chance", *Journal of Machine Learning Research*, vol. 11, pp. 2837–2854, 2010.

[10] A. Rosenberg and J. Hirschberg, "V-measure: A conditional entropy-based external cluster evaluation measure", in *Proceedings of the 2007 Joint Conference on Empirical Methods in Natural Language Processing and Computational Natural Language Learning (EMNLP-CoNLL)*, 2007, pp. 410–420.

[11] T. Chen, S. Kornblith, M. Norouzi, and G. Hinton, "A simple framework for contrastive learning of visual representations", in *International Conference on Machine Learning (ICML)*, 2020.

[12] A. van den Oord, Y. Li, and O. Vinyals, *Representation learning with contrastive predictive coding*, arXiv preprint arXiv:1807.03748, 2018. arXiv: 1807.03748 [cs.LG].

[13] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention is all you need", in *Advances in Neural Information Processing Systems (NeurIPS)*, 2017, pp. 5998–6008.

[14] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding", in *Proceedings of NAACL-HLT*, 2019, pp. 4171–4186.

[15] J. Su, M. Ahmed, Y. Lu, S. Pan, W. Bo, and Y. Liu, "RoFormer: Enhanced transformer with rotary position embedding", *Neurocomputing*, vol. 568, p. 127 063, 2024. DOI: 10.1016/j.neucom.2023.127063

[16] H. K. Mardia, "New techniques for the deinterleaving of repetitive sequences", *IEE Proceedings F (Radar and Signal Processing)*, vol. 136, no. 4, pp. 149–154, 1989. DOI: 10.1049/ip-f-2.1989.0025

[17] D. J. Milojević and B. M. Popović, "Improved algorithm for the deinterleaving of radar pulses", *IEE Proceedings F (Radar and Signal Processing)*, vol. 139, no. 1, pp. 98–104, 1992. DOI: 10.1049/ip-f-2.1992.0012

[18] S. Bedoire, "Offline direction clustering of overlapping radar pulses from homogeneous emitters", Host company Saab; DBSCAN-style clustering on pulse directions with simulated scenarios, Master's thesis, KTH Royal Institute of Technology, Stockholm, Sweden, 2022.

[19] P. Notaro, M. Paschali, C. Hopke, D. Wittmann, and N. Navab, *Radar emitter classification with attribute-specific recurrent neural networks*, arXiv preprint arXiv:1911.07683, 2019. arXiv: 1911.07683 [eess.SP].

[20] T. Jonsson, "Classification of Radar Emitters using Semi-Supervised Contrastive Learning", Host company Saab AB; supervised learning experiments on pulse data from the OpenModesty scenario simulator, Master's thesis, KTH Royal Institute of Technology, Stockholm, Sweden, 2023.

[21] K. Coleman, "Dataset drift in radar warning receivers: Out-of-distribution detection for radar emitter classification using an RNN-based deep ensemble", UPTEC F 23041, 30 hp; simulator data from Saab AB and drift/OOD analysis for emitter classification, Master's thesis, Uppsala University, Uppsala, Sweden, 2023.

[22] R. J. G. B. Campello, D. Moulavi, A. Zimek, and J. Sander, "Hierarchical density estimates for data clustering, visualization, and outlier detection", *ACM Transactions on Knowledge Discovery from Data*, vol. 10, no. 1, 2015. DOI: 10.1145/2733381

[23] L. McInnes, J. Healy, and S. Astels, "hdbscan: Hierarchical density based clustering", *The Journal of Open Source Software*, vol. 2, no. 11, p. 205, 2017. DOI: 10.21105/joss.00205

[24] P. Khosla, P. Teterwak, C. Wang, A. Sarna, Y. Tian, P. Isola, A. Maschinot, C. Liu, and D. Krishnan, "Supervised contrastive learning", in *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.

[25] X. Guo, L. Gao, X. Liu, and J. Yin, "Improved deep embedded clustering with local structure preservation", in *Proceedings of the 26th International Joint Conference on Artificial Intelligence (IJCAI)*, 2017, pp. 1753–1759. DOI: 10.24963/ijcai.2017/243

[26] M. Caron, P. Bojanowski, A. Joulin, and M. Douze, "Deep clustering for unsupervised learning of visual features", in *Proceedings of the European Conference on Computer Vision (ECCV)*, 2018, pp. 132–149.

[27] M. Caron, I. Misra, J. Mairal, P. Goyal, P. Bojanowski, and A. Joulin, "Unsupervised learning of visual features by contrasting cluster assignments", in *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.

[28] Y. Li, P. Hu, Z. Liu, D. Peng, J. T. Zhou, and X. Peng, "Contrastive clustering", in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 35, 2021, pp. 8547–8555. DOI: 10.1609/aaai.v35i10.17037

[29] F. Schroff, D. Kalenichenko, and J. Philbin, "FaceNet: A unified embedding for face recognition and clustering", in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2015, pp. 815–823. DOI: 10.1109/CVPR.2015.7298682

[30] J. Lee, Y. Lee, J. Kim, A. Kosiorek, S. Choi, and Y. W. Teh, "Set transformer: A framework for attention-based permutation-invariant neural networks", in *Proceedings of the 36th International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 97, PMLR, 2019, pp. 3744–3753.

[31] G. Zerveas, S. Jayaraman, D. Patel, A. Bhamidipaty, and C. Eickhoff, "A transformer-based framework for multivariate time series representation learning", in *Proceedings of the 27th ACM SIGKDD Conference on Knowledge Discovery and Data Mining (KDD)*, 2021, pp. 2114–2124. DOI: 10.1145/3447548.3467401

[32] P. Shaw, J. Uszkoreit, and A. Vaswani, "Self-attention with relative position representations", in *Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (NAACL-HLT)*, 2018, pp. 464–468. DOI: 10.18653/v1/N18-2074

[33] C. Ying, T. Cai, S. Luo, S. Zheng, G. Ke, D. He, Y. Shen, and T.-Y. Liu, "Do transformers really perform bad for graph representation?", in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 34, 2021, pp. 28 877–28 888.

[34] B. Heo, S. Park, D. Han, and S. Yun, "Rotary position embedding for vision transformer", in *Proceedings of the European Conference on Computer Vision (ECCV)*, 2024, pp. 289–305. DOI: 10.1007/978-3-031-72684-2_17

[35] R. Caruana, "Multitask learning", *Machine Learning*, vol. 28, no. 1, pp. 41–75, 1997. DOI: 10.1023/A:1007379606734

[36] J. Yang, D. Parikh, and D. Batra, "Joint unsupervised learning of deep representations and image clusters", in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 5147–5156.

[37] I. Loshchilov and F. Hutter, "Decoupled weight decay regularization", in *International Conference on Learning Representations (ICLR)*, 2019.

[38] V. Jakobsson, "Offline radar pulse track association with deinterleaving errors", Host company SAAB; time-series clustering for track fragments under imperfect deinterleaving, Master's thesis, KTH Royal Institute of Technology, Stockholm, Sweden, 2024.

APPENDIX A

Additional Results

Placeholder paragraph.

APPENDIX B

Implementation Details

Placeholder paragraph.

APPENDIX C

Notation Reference

Placeholder paragraph.

APPENDIX D

Probabilistic Reading of the Contrastive Loss

This appendix expands the supervised contrastive kernel of Equation (4.25). The main text states the loss used in training; the derivation below is optional background.

For each pulse $i \in \mathcal{B}^+$, define the categorical distribution over its candidates

$$P_{\boldsymbol{\theta}}(a \mid i) \;=\; \frac{\exp(s_{ia}/\tau)}{Z_i}, \qquad Z_i \;=\; \sum_{a' \in A(i)} \exp(s_{ia'}/\tau), \qquad a \in A(i), \tag{D.1}$$

which is the Boltzmann distribution with energy $-s_{ia}$ and temperature τ on $A(i)$. Substituting Equation (D.1) into Equation (4.25) yields the per-pulse cross-entropy between the empirical positive distribution (uniform on $P(i)$) and the model $P_{\boldsymbol{\theta}}(\cdot \mid i)$,

$$\mathcal{L}_{\mathrm{SC}} \;=\; -\sum_{i \in \mathcal{B}^+} \frac{1}{|P(i)|} \sum_{p \in P(i)} \log P_{\boldsymbol{\theta}}(p \mid i). \tag{D.2}$$

That is minus the average log-evidence assigned to the positives, as if $P(i)$ were observations from a latent positive class.

Applying $\log(x/y) = \log x - \log y$ to Equation (4.25) separates attraction and repulsion:

$$\mathcal{L}_{\mathrm{SC}} \;=\; \underbrace{\sum_{i \in \mathcal{B}^+} \log Z_i}_{\text{log-partition (repulsion)}} \;-\; \underbrace{\sum_{i \in \mathcal{B}^+} \frac{1}{|P(i)|} \sum_{p \in P(i)} \frac{s_{ip}}{\tau}}_{\text{mean positive similarity (attraction)}} . \tag{D.3}$$

The first term grows with similarity to every candidate; its gradient $\partial \log Z_i / \partial s_{ia} = P_{\boldsymbol{\theta}}(a \mid i)/\tau$ concentrates on hard negatives, i.e. candidates with large Boltzmann weight under Equation (D.1) despite a different label. The second term rewards alignment with the positives, each contributing $1/(\tau|P(i)|)$ to the gradient.

The prefactor $1/|P(i)|$ turns the inner sum into an arithmetic mean, or equivalently the log of a geometric mean,

$$-\frac{1}{|P(i)|} \sum_{p \in P(i)} \log P_{\boldsymbol{\theta}}(p \mid i) \;=\; -\log \left(\prod_{p \in P(i)} P_{\boldsymbol{\theta}}(p \mid i) \right)^{1/|P(i)|} . \tag{D.4}$$

Without it, the inner sum would be the log-likelihood of $|P(i)|$ independent positive observations and would scale with the size of $P(i)$. Dividing by $|P(i)|$ makes each pulse's contribution intensive in its positive-set size. That is why the same kernel stays

well-behaved on both branches, where $|P(i)|$ ranges from a few emitter matches inside one window to many hundreds of mode matches across the batch.